NNStudio — Blueprints

A guided tour through the source code, written for humans.
Each chapter links directly to the relevant file. Open the file alongside this doc in VS Code (`Ctrl+\` to split editor).

> **Reading order:** follow the chapters top-to-bottom on a first pass.
> Later, use this as a reference/sitemap — every concept links back to the implementation.

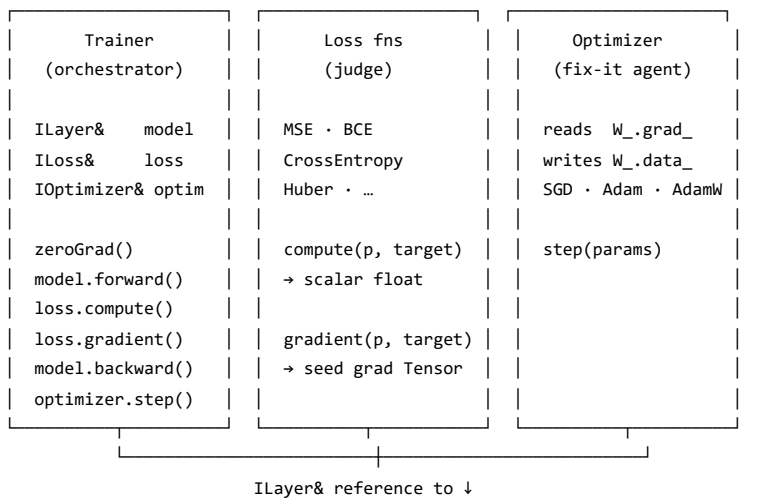
The Story — How All Pieces Plug Together

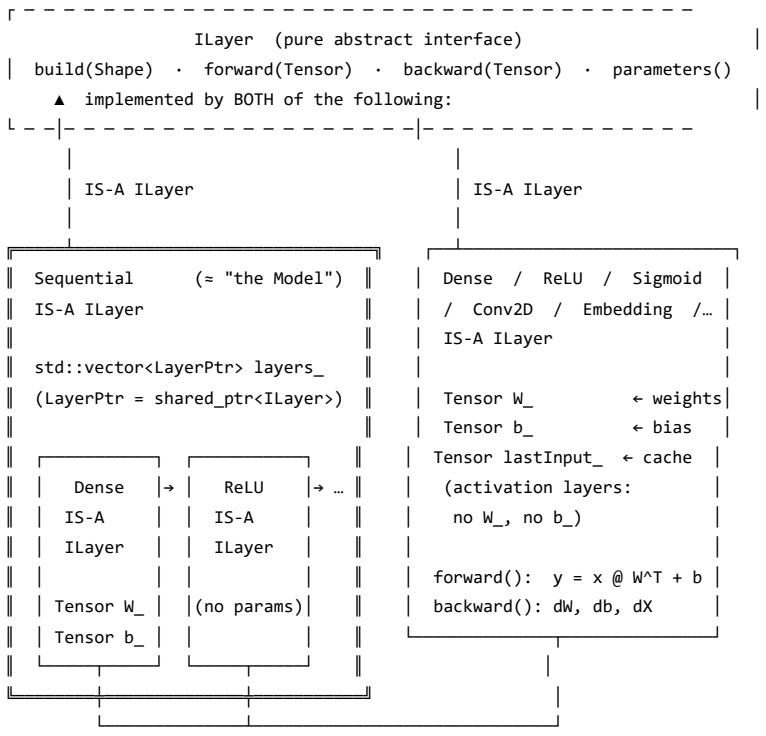
Each piece in this engine answers one question left open by the piece before it.

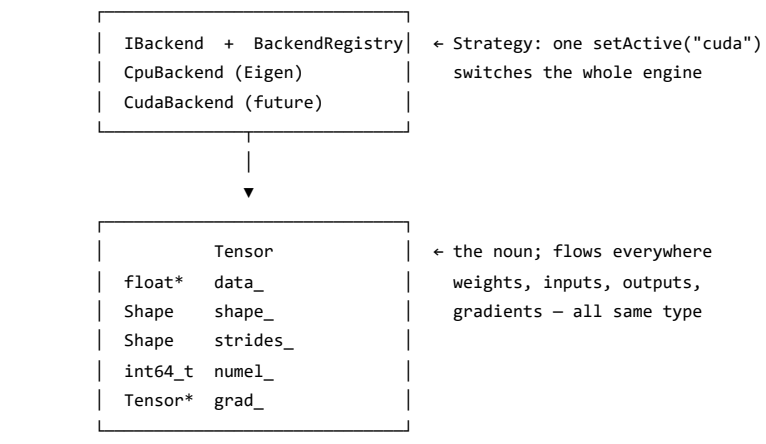
```
"I need data to flow."
    └─ Tensor                  the noun — a smart flat float array
       shape + strides + grad  describes any data: weights, inputs, outputs, gradients

"But who does the math on that data?"
    └─ IBackend                the blueprint (Strategy interface)
       CpuBackend              one answer: Eigen on CPU
       CudaBackend             another answer: cuBLAS on GPU  (future)
       BackendRegistry         the switchboard — backend() gives you whichever is active

"But who organises a meaningful calculation?"
    └─ ILayer / Dense          one processing step
       owns:  W_, b_           learnable Tensors (requires_grad = true)
       does:  forward(x)  →  y = x @ W^T + b   (calls backend)
              backward(dY) →  dW, db, dX        (calls backend, fills W_.grad_)

"But who chains layers into a network?"
    └─ Sequential              ordered list: output of layer N → input of layer N+1
       ComputeGraph            DAG version: records ops for full autograd  (Chapter 7)

"But who judges if the output is correct?"
    └─ Loss (MSE, BCE, ...)    compares prediction vs target → scalar float
                               also produces seed gradient dLoss/dPrediction

"But who fixes the weights?"
    └─ Optimizer (SGD, Adam)   reads W_.grad_ for every parameter
                               nudges W_ downhill on the loss surface

"But who runs the loop?"
    └─ Trainer                 zeroGrad → forward → loss → backward → step
                               repeats for every batch, every epoch
                               fires callbacks so the UI can watch live

"But how do we know it all works?"
    └─ XOR test                4 samples, simplest non-linear problem
                               passes when loss < 0.05 and all 4 predictions correct
                               = proof the whole chain collaborates correctly
```

Or as a class diagram:

The three tools that OPERATE ON the model — hold references, not ownership:







Architectural Invariants

> **How to use this list.**
> Whenever a conversation shifts toward designing or reviewing a *published interface* — anything a plugin author will implement, anything crossing a phase boundary, anything going into PLUGIN-SDK.md or an `IXxx.h` header — stop and verify each item below. The list exists because the dangerous moment is when everything *looks* fine.

#	Invariant	Rationale
I-1	Every published interface (ILayer, IActivation, IBackend, Plugin SDK) must be **reentrant** — safe to call concurrently on the same object with distinct caller-owned state	Phase 2 users will run models in web servers and GPU backends; silent wrong-gradient failures under concurrency are unacceptable
I-2	No published interface shall hold **mutable state between a call-pair** (e.g. forward/backward) inside the implementing object	Same reason as I-1; caller-owned context (ADR-020 Option C) is the approved pattern
I-3	**Silent failure modes** (wrong result, no crash, no assertion) are never acceptable in a published interface; prefer `Result<T>` + assert over undefined behaviour	Wrong gradients that only appear under load are the hardest possible bugs to diagnose
I-4	The **Phase 2 boundary** is the highest-risk design gate: any interface a plugin author will implement must be reviewed against I-1 through I-3 before the first line of SDK documentation is written	Post-publication API breaks have downstream cost; pre-publication review has near-zero cost
I-5	`Tensor` copy = **reference-count increment only** — no float data copied unless `.clone()` is called explicitly	Performance assumption underlying all interface designs; if violated, every interface signature discussion changes

This list is a living document. Add an entry whenever a design review reveals a property that must hold across the entire codebase.

Chapter → Source Navigator

Every concept in this document maps to a specific file. Use this table as a quick entry point before reading a chapter; use Appendix A for the full annotated tree.

Chapter / Section	Concept	Interface header(s)	Default implementation	Tests
Ch.1	Tensor memory model	`include/core/Tensor.h`	`src/core/Tensor.cpp`	`tests/core/test_tensor.c`
Ch.2	Backend / compute dispatch	`include/core/IBackend.h` · `BackendRegistry.h`	`src/builtin/backends/CpuBackend.cpp`	—
Ch.3	Layer contract · Dense · Activations	`include/core/Layer.h` · `IActivation.h` · `include/builtin/layers/Dense.h` · `include/builtin/activations/`	`src/builtin/layers/Dense.cpp` · `src/builtin/activations/`	`tests/builtin/test_layer` `test_activations.cpp`
Ch.4	Sequential pipeline · ComputeGraph	`include/core/Layer.h` · `ComputeGraph.h`	`src/core/Layer.cpp` · `src/core/ComputeGraph.cpp`	`tests/core/test_compute_`
Ch.5	Loss functions	`include/core/ILoss.h` · `include/builtin/losses/Losses.h`	`src/builtin/losses/Losses.cpp`	`tests/builtin/test_losse`
Ch.6	Optimizers	`include/core/IOptimizer.h` · `include/builtin/optimizers/Optimizers.h`	`src/builtin/optimizers/Optimizers.cpp`	`tests/builtin/test_optim`
Ch.7	Trainer loop	`include/core/Trainer.h`	`src/core/Trainer.cpp`	—
Ch.8	XOR milestone	—	—	`tests/core/test_trainer_`

Chapter / Section	Concept	Interface header(s)	Default implementation	Tests
§10.1	Activation ADR-020 (functors + adapter)	`include/builtin/activations/Functors.h` · `FnLayer.h`	`src/builtin/activations/Functors.cpp`	`tests/builtin/test_activ`
§10.2	Full layer suite (Norm · Embedding · MHA · Conv2D)	`include/builtin/layers/`	`src/builtin/layers/`	`tests/builtin/test_layer` `test_conv2d.cpp` · `test_e`
§10.3–4	Full loss + optimizer suites	same as Ch.5–6	same	same
§10.5	Checkpoint · EarlyStopping	`include/core/Checkpoint.h` · `EarlyStopping.h`	`src/core/Checkpoint.cpp`	`tests/core/test_checkpoi` `test_early_stopping.cpp`
§10.6	`torch_compat` · CompatibilityChecker	`include/torch_compat.h` · `include/core/CompatibilityChecker.h`	`src/core/CompatibilityChecker.cpp`	`tests/compat/test_torch_` `tests/core/test_compat.c`
§10.7	Plugin SDK · Trust architecture	`include/plugin-api/nnstudio_plugin.h` · `include/plugin-api/trust/` · `plugin-api/`	`src/plugin-api/`	`tests/plugin-api/test_pl` · `test_trust_store.cpp` · `test_trust_verifier.cpp`
§10.8	Reference plugins (BPE · ExampleActivation)	`plugins/bpe_tokenizer/` · `plugins/example_activation/`	same	`tests/plugin-api/test_bp`
§10.9	Python bridge · Keras façade · runners	`python-bridge/bindings/module.cpp` · `python-bridge/nnstudio/`	same	—

Chapter 1 — The Memory Model (`Tensor`)

File: `core/include/nnstudio/core/Tensor.h`

Overview (plain language)

Physically it is a flat, one-dimensional array of floats in memory (a `std::vector<float>` under the hood).
Logically it can represent any multi-dimensional structure — a vector, a matrix, a 3D cube — by using `shape_` and `strides_` as a navigation map on top of that flat array. It is, in the most literal sense, a one-row matrix that happens to describe itself as multi-dimensional through those navigation parameters.

In addition to the data itself, `Tensor` carries two important companions:

- `grad_` — the **gradient**: a sibling array of the same shape that accumulates, during training, the answer to the question *"by how much would the loss change if this particular float were nudged slightly?"* It is the engine's memory of blame, built up step by step during each backward pass.
- `numel_` — a **cached element count**: purely a computational shortcut. Multiplying a large shape every time it is needed would be expensive; `numel_` stores the result once and saves those cycles.

A `Tensor` instance is intentionally role-neutral — the instances (instance variables) of the very same class may represent weights, biases, layer inputs, layer outputs, and gradients. Which role it plays is determined entirely by context (eg. class who owns it (typically some ILayer implementation, see below), how it is named, whether `requires_grad` is set).

What Tensor is (and isn't)

`Tensor` is a **generic N-dimensional float array**.
It is used for *everything* — weights, biases, inputs, outputs, and gradients all live in separate `Tensor` instances. Role is determined by context, not by type.

It is **not** a compute engine. It holds data. All arithmetic is delegated to `IBackend` (see Chapter 2).

The four key private members

```
data_    — flat float[] on the heap, owned via shared_ptr
shape_   — e.g. {3, 2} = 3 rows, 2 columns
strides_ — how many floats to skip per dimension to navigate data_
numel_   — cached product of shape (= total element count)
grad_    — optional sibling Tensor holding the gradient (null until needed)
```

What are strides?

Strides are the step sizes you add to your current array index to move one unit in each dimension.
For a 2D matrix encoded as one flat vector, there are exactly two stride numbers — think of them as questions:

- `strides_[0]` — *"how many elements do I skip in `data_[]` to move down by one **row**?"*
- `strides_[1]` — *"how many elements do I skip in `data_[]` to move right by one **column**?"*

The formula for any element at logical index `[row, col]`:

```
offset = row × strides_[0] + col × strides_[1]
```

Example — shape `{3, 2}`, strides `{2, 1}`:

```
data_ = [ 10,  20,  30,  40,  50,  60 ]
index    [0]  [1]  [2]  [3]  [4]  [5]

[row=1, col=0] → 1×2 + 0×1 = 2 → data_[2] = 30  ✓
[row=1, col=1] → 1×2 + 1×1 = 3 → data_[3] = 40  ✓
```

The rule for row-major (C-order) strides: `strides_[k] = product of all dims after k`.
Shape `{3,2}` → strides `{2,1}`. Shape `{4,3,2}` → strides `{6,2,1}`.
Strides have **nothing to do with computation** — they are purely a navigation map: "given a logical index, which float in `data_[]` is that?" Every computation result is always a brand-new `Tensor` with its own `data_[]`.

How that lives in memory — Dense 2→3 example

A `Dense(2→3)` layer has 2 inputs and 3 neurons.

Input `x`, shape `{1, 2}` — one sample, two features:

```
data_ = [ 0.6,  0.4 ]
          x₁    x₂
strides_ = {2, 1}  — skip 2 floats per row, 1 float per column
```

Weight matrix `W`, shape `{3, 2}` — 3 neurons, each with 2 weights:

```
data_ = [ w₀₀, w₀₁,   ← neuron 0's weights (its "sensitivity" to each input)
          w₁₀, w₁₁,   ← neuron 1's weights
          w₂₀, w₂₁ ]  ← neuron 2's weights
strides_ = {2, 1}
```

Each row = one neuron.

Output `y = x @ W^T + b`, shape `{1, 3}`:

```
y[0] = 0.6·w₀₀ + 0.4·w₀₁ + b₀   ← neuron 0's output
y[1] = 0.6·w₁₀ + 0.4·w₁₁ + b₁   ← neuron 1's output
y[2] = 0.6·w₂₀ + 0.4·w₂₁ + b₂   ← neuron 2's output
```

Each output is exactly the "weighted sum of all inputs plus bias" that defines a neuron.

> **Implementation:**
> `Dense.cpp — forward()`
> `BackendRegistry.h — backend()`

Batching — why the first dimension exists

Running one sample at a time is slow. Instead we stack `B` samples into one tensor, shape `{B, inF}`:

```
data_ = [ x₁⁽⁰⁾, x₂⁽⁰⁾,   ← sample 0
          x₁⁽¹⁾, x₂⁽¹⁾,   ← sample 1
          ...             ]
```

The matmul `x @ W^T` produces `{B, outF}` — all `B` outputs in one Eigen (or CUDA) call.
That is the entire reason layers are written as matrix operations rather than loops over neurons.

In the XOR test batch size is always 1, so shapes flow as: `{1,2} → {1,4} → {1,1}`.

Transpose is free — strides without copying

`Dense::forward` needs `W^T`. Instead of copying the data, we just swap `strides_`:

```
W   shape={3,2}  strides_={2,1}   ← read data as rows of 2
W^T shape={2,3}  strides_={1,2}   ← same data_, different navigation
```

`shared_ptr<float[]>` means both views share the same heap allocation. Not a single float moves.
This is the entire purpose of storing `strides_` separately from `shape_`.

The gradient tensor

After backpropagation, `W.grad_` has shape `{3, 2}` — identical to `W`.
Each element `grad_[i][j]` answers: *"if I increase `W[i][j]` by a tiny amount, how much does the loss change?"*

The optimizer (`Adam`, `SGD`) then subtracts a scaled version of `grad_` from `W.data_`.
Because both have the same shape, Adam can do this as a single flat loop over `numel_` floats.

> See also: Chapter 7 — The Training Loop for where zeroGrad/accumulateGrad/step fit together.

§1.x — Foundation types: `Device`, `DType`, `Result<T>`

Three small enums/types serve as the vocabulary shared by every other file in the engine. They live in `core/include/nnstudio/core/` and are included transitively by almost everything.

`Device` (Device.h) — a `uint8_t` enum tag stored on every Tensor declaring *where its data lives*: `CPU` (main memory, Phase 1 default), `CUDA` (GPU device memory, Phase 2+), `Quantum` (Phase 6 stub). Device is a property of **data**, not computation — a `DeviceMismatch` error fires before any arithmetic if the active Backend and the Tensor disagree.

`DType` (DType.h) — the element type of a Tensor: `Float32` (training, always supported), `Float16` (inference compression, GPU), `Int8` (quantized export), `Int32` (index/token-ID tensors), `Bool` (masks). Phase 1 computes in `Float32` only; the others are reserved for CUDA and quantization phases. `dtypeBytes(d)` drives all stride and memory-size calculations; `dtypeName(d)` produces the stable string used in ONNX and `.nns` serialization.

`Result<T>` (Result.h) — the error-handling contract for the entire engine, replacing exceptions (see ADR-011). The engine is compiled with `-fno-exceptions` and `-fno-rtti` because it will be linked into plugin shared libraries; C++ exceptions crossing shared-library boundaries are undefined behaviour in the C ABI. `Result<T>` holds either a `T` value (success) or an `Error` (code + message string). `Result<void>` / `VoidResult` carries success-or-error with no payload. The `NN_TRY(expr)` macro propagates an error upward immediately — analogous to Rust's `?` — and is the standard idiom in any function that calls multiple fallible operations in sequence.

Chapter 2 — Who Does the Math? (`IBackend` + `CpuBackend`)

Files:

`core/include/nnstudio/core/IBackend.h`

`core/include/nnstudio/core/BackendRegistry.h`
`backends/cpu/CpuBackend.cpp`

Overview (plain language)

Then we have the `Backend` — the interface to our pocket-calculator chip.
It is a wrapper providing the fundamental mathematical operations needed when computing neuron functions in higher-level classes. The two core operations are **matrix multiplication** and **transposition**; it also provides elementwise arithmetic, reductions (`sum` , `mean`), and reshaping.

The reason this abstraction exists is portability: the same operations can be routed to a CPU implementation today, a CUDA GPU tomorrow, or a quantum accelerator the day after — without any layer ever needing to know which one is active. The currently active implementation is managed by `BackendRegistry` , a singleton registry of named backend instances. A single `setActive("cuda")` call is all that is needed to switch the entire engine from CPU to GPU math.

Both `Tensor` and `Backend` / `BackendRegistry` are deliberately ignorant of neural network concepts. They have no notion of neurons, forward propagation, or training. Their design is strictly NN-utilitarian: they provide the data and operations foundation that higher-level NN classes can be built on top of, independent of the underlying hardware.

`Tensor` holds data. `IBackend` computes on it.
`BackendRegistry` is a singleton that holds the active backend.
`backend()` (free function in `BackendRegistry.h`) always returns the active one — so no layer ever mentions `CpuBackend` directly. Switching to CUDA is a single `setActive("cuda")` call.

The Eigen row/col-major trick

Eigen stores matrices column-major (memory goes down columns first).
Our tensors are row-major (memory goes across rows first, like C arrays).

Key identity used throughout `CpuBackend::matmul` :

```
(A_rowmajor @ B_rowmajor)  ≡  interpreted col-major  =  (B^T @ A^T)
```

So for `C[M,N] = A[M,K] @ B[K,N]` we map to Eigen as:

```
Eigen sees A as K×M col-major  =  A^T
Eigen sees B as N×K col-major  =  B^T
Eigen writes C as N×M col-major  =  C^T
C^T = B^T @ A^T   ✓  (mathematically identical to C = A @ B)
```

`noalias()` tells Eigen the output buffer doesn't overlap any input — skips a defensive internal copy.

> **Reference:** For a fully annotated specification of every `IBackend` virtual method — mathematical notation, numeric micro-examples, per-layer CUDA-readiness analysis, strategy for adding new methods without breaking existing backends, and a compatibility matrix across all planned backends — see Appendix D — IBackend Vtable Reference.

Chapter 3 — A Layer's Contract (`ILayer` + `Dense`)

Files:
`core/include/nnstudio/core/Layer.h`
`core/include/nnstudio/core/layers/Dense.h`
`core/src/layers/Dense.cpp`

Overview (plain language)

`ILayer` is the first interface that is genuinely about neural networks.
It represents a single computational step in a network — a class that owns a collection of learnable parameter variables of the type `Tensor` (weights `W` and bias `b`) and knows how to command the `BackendRegistry` to perform calculations on them.

When looking at the functions of such a class, apart from initialization (`construct` , `build`) and a housekeeping reset (`zeroGrad`), it exposes two key calculation functions:

- `forward(x)` — the forward propagation step: takes the input activation vector (the outputs of the previous layer), runs the layer's mathematical formula, and returns the output activation vector for the next layer.
- `backward(gradOut)` — the reverse flow used during training: receives the error gradient arriving from the layer above, uses it to compute how wrong each weight and bias was (`dW` , `db`), accumulates those into the parameter `grad_` fields, and returns the gradient signal `dX` to pass further back to the layer below. Structurally it mirrors `forward()` but runs in the opposite direction.

Architectural notes on `ILayer` and its concrete types:

- `ILayer` is only the interface definition for a NN layer. However in a NN, there might be multiple types of layers, differing by the architecture of the "interconnections" and calculations (which inputs/previous layer neuron outputs are used and how, and which mathematical functions are applied to them during forward/backward propagation operations).
- `Dense` is the most fundamental concrete implementation of `ILayer` — the "no structure assumed" baseline. It is a **fully-connected** layer: every single input value is connected to every single output neuron with its own independent learned weight. All connections are always computed, even those whose weights are near zero; skipping connections with insignificant weights requires explicit post-training pruning, which is a separate step not performed here.
- There are also other implementations of `ILayer` apart from `Dense` : `ReLU` / `Sigmoid` / `Tanh` (parameterless activations — a fixed mathematical gate applied elementwise, no learned weights), `Conv2D` (local spatial filter — each output connects only to a small neighbourhood of the input, not all of it; key for image/signal data), `BatchNorm` (normalisation — 2 learned scale factors per feature; keeps activations in a well-behaved range during training), `Dropout` (randomly zeroes some activations at train time — a regularisation technique), `Embedding` (integer token index → learned float vector via a direct lookup table — typical first layer in text/language networks). A full comparison table is in §3.7.

A note on neurons: there is no `Neuron` class, no `Neuron` object, no abstract `INeuron` interface anywhere in this engine. This is correct and intentional. A neuron in a `Dense` layer has no individual identity, no functionality of its own, and no biological meaning beyond a convenient metaphor. What we call "neuron k" is simply a set of related indices in the weight matrix `W` : row `k` of `W` holds that neuron's weights, element `k` of `b` holds its bias, and element `k` of the output vector holds its activation. The neuron exists only as a virtual address (an index into the `Tensor` -typed variables in an `ILayer` implementing class instance such as `Dense`) — an implicit `[row, *]` slice of a matrix — never as a stored object.

Layer vs Backend — what is each responsible for?

	Responsibility
Backend	Raw tensor math: `matmul` , `add` , `exp` , `sum` … pure arithmetic, no concept of neurons
Layer	A meaningful NN operation: holds `W` and `b` , uses Backend to run its formula, computes gradients
Optimizer	Reads those gradients and **updates** the weight values — the Layer never touches its own weights

The Layer is the *"who is to blame?"* accountant. The Optimizer is the *"fix it"* agent.

3.1 Lifecycle — the contract every layer must honour

```
construct → build(inputShape) → forward(x) → backward(gradOut) → zeroGrad()
```

Step	Method	What happens
1	**construct**	Store hyper-params (e.g. `outFeatures_`). Nothing allocated — `inFeatures` not yet known.
2	**build(inputShape)**	Now we know `inFeatures` → allocate `W` and `b` . Idempotent: same shape twice = no-op.
3	**forward(x)**	Run the NN formula. Save `lastInput_` — backward will need it. Return output tensor.
4	**backward(gradOut)**	Receive `dL/dY` from the layer above. Accumulate `dW` , `db` into `grad_` . Return `dX` downward.
5	**zeroGrad()**	Reset all `grad_` fields to 0. Trainer calls this before every training step.

> The layer accumulates gradients but does **not** update weights — that is the Optimizer's job.

3.2 What are `B`, `inF`, `outF`?

Start with **one sample**: say a 28×28 grayscale image flattened to 784 pixel values.

```
inF  (inFeatures) = 784   ← how many numbers describe ONE sample entering this layer
                            • Layer 0: the raw input size you provide
                            • Any other layer: equals the previous layer's outF

outF (outFeatures) = 128  ← you chose this; Dense(128) means "produce 128 numbers per sample"
                            • Becomes inF of the next layer
                            • One number per neuron in this layer

B (batch size) = 32       ← samples processed simultaneously
                            • Each sample is one ROW in the input matrix
                            • The matmul handles all rows in one call
                            • Per-sample math is identical; batch just amortises the call cost
```

So the tensor flowing between layers is:

```
x: [32, 784]   ← 32 rows (samples), 784 columns (features per sample)
    ^B      ^inF

y: [32, 128]   ← 32 rows of output, one per input sample
    ^B      ^outF
```

3.3 What is Dense? — from one neuron to a matrix

One neuron has `inF` incoming connections, each with its own weight, plus a bias:

```
output = w₁·x₁ + w₂·x₂ + … + w_inF·x_inF + b
       = dot(w_row, x) + b
       = one scalar
```

Dense = `outF` such neurons, all reading the same input:

```
neuron 1:    y₁     = dot(w_row₁,    x) + b₁
neuron 2:    y₂     = dot(w_row₂,    x) + b₂
             …
neuron outF: y_outF = dot(w_row_outF, x) + b_outF
```

Stack all weight rows into a matrix `W` of shape `[outF, inF]`:

```
ONE sample:   y = W @ x + b          [outF]   = [outF,inF] @ [inF]   + [outF]
BATCH:        Y = X @ W^T + b        [B,outF] = [B,inF]   @ [inF,outF] + [outF]
```

(The transpose `W^T` makes shapes align: `[B,inF] @ [inF,outF] = [B,outF]`.)

"Fully connected" / "Dense": every input x_i connects to every neuron j with its own independent weight w_{ji}.
Total parameters: `outF × inF` weights + `outF` biases = `outF × (inF + 1)`.

> **Near-zero weights are still computed.** Dense always runs all `outF×inF` multiply-adds.
> A weight of ~0 means "this connection barely matters" — but the hardware still multiplies by it.
> Skipping such connections requires post-training **weight pruning / sparsification** (not implemented here).

3.4 Forward pass: `Y = X @ W^T + b`

```
X       [B, inF]       raw data or previous layer's output — each row = one sample
W       [outF, inF]    weight matrix — each ROW = one neuron's weight vector
W^T     [inF, outF]    transposed so matmul shapes align with the batch input
Y       [B, outF]      one output row per sample → feeds into the next layer
b       [outF]         one bias per output neuron, broadcast over the batch
```

`lastInput_ = X` is saved here — the backward pass needs it to compute `dW`.
Without storing it, we'd have to re-run forward, doubling the work.

3.5 Backward pass: three gradients from one incoming signal

`gradOut` = `dL/dY` arrives from the layer **above** — shape `[B, outF]`.
We must produce:

Gradient	Formula	Shape	Meaning	Destination
`dW`	`gradOut^T @ X`	`[outF, inF]`	how wrong was each weight?	stored in `W_.grad_`
`db`	`sum(gradOut, dim=0)`	`[outF]`	how wrong was each bias?	stored in `b_.grad_`
`dX`	`gradOut @ W`	`[B, inF]`	what do I blame the layer below for?	**returned** downward

Shape check for `dW`: `gradOut^T` is `[outF, B]` × `X` is `[B, inF]` = `[outF, inF]`. Same shape as `W`. ✓

`accumulateGrad()` **adds** into `grad_` rather than replacing it — supports running multiple forward/backward passes before calling `optimizer.step()` once (gradient accumulation for memory saving).

`dX` is the signal the layer below uses to compute *its own* `dW`.

3.6 How Dense plugs into the Tensor → Backend → Layer system

```
User code
  └ Dense::forward(X)                  ← concrete ILayer subclass
       └ B().transpose(W_.tensor)       ← fetches active Backend via free function B()
       └ B().matmul(X, Wt)              ← CpuBackend → Eigen GEMM on raw float* pointers
            └ Tensor::data()            ← returns pointer into flat std::vector<float>
```

The layer gives **NN meaning** to the numbers. The Backend does the math. The Tensor holds the bytes.
Nothing below the Layer level knows what "weights" or "neurons" are.

Memory note: `transpose()` only swaps `strides_[]` — no data is copied.
The same `float*` block is read by Eigen with a different stride interpretation.

3.7 Dense is one of many ILayer types

Class	Namespace	Has parameters?	Connectivity	Mental box
Dense	`nnstudio::layers`	Yes: `W_`, `b_`	Every input → every output	Most general; most expensive
ReLU	`nnstudio::activations`	No	1-to-1 elementwise	`y = max(0,x)` — non-linearity, zero params
LeakyReLU	`nnstudio::activations`	No (but has `alpha_` hyper-param)	1-to-1 elementwise	Like ReLU but negative side scales by α

Class	Namespace	Has parameters?	Connectivity	Mental box
Sigmoid	`nnstudio::activations`	No	1-to-1 elementwise	Squash to (0,1); output = probability
TanhAct	`nnstudio::activations`	No	1-to-1 elementwise	Squash to (−1,1); zero-centred variant
Softmax	`nnstudio::activations`	No	All inputs → all outputs	Normalise to probability distribution; NOT elementwise
GELU	`nnstudio::activations`	No	1-to-1 elementwise	Smooth probabilistic gate; used in Transformers
Conv2D	`nnstudio::layers`	Yes: small kernel	Local patch → one output	Fewer weights; shared spatially
BatchNorm	`nnstudio::layers`	Yes: γ, β (2 per feature)	1-to-1 with statistics	Keeps activations well-scaled
Dropout	`nnstudio::layers`	No	Randomly zeros activations	Regularisation; train/eval differ
Embedding	`nnstudio::layers`	Yes: lookup table	Integer index → float vector	Token ID → dense representation

`Dense` lives in `nnstudio::builtin::layers::` . Activations also live in `nnstudio::builtin::layers::` . Both extend the same `nnstudio::ILayer` . See §3.8 for the inheritance hierarchy.

3.8 Activation functions — `IActivation` , functors, and `ActivationBase`

Files:

`include/core/IActivation.h`
`include/builtin/layers/ActivationFunctors.h`
`include/builtin/layers/Activations.h`
`include/builtin/layers/ActivationsFnLayer.h`
`src/builtin/layers/ActivationFunctors.cpp`
`src/builtin/layers/Activations.cpp`
`tests/builtin/test_activations.cpp`

The two-tier design (ADR-020)

Activations now live on two tiers:

```
Tier 1 — pure stateless functor  (nnstudio::core::IActivation)
  No mutable state. Can be shared across threads and graph nodes.
  Implemented by: ReLUFn, LeakyReLUFn, SigmoidFn, TanhFn, SoftmaxFn, GELUFn

Tier 2 — ILayer adapter  (wraps one Tier-1 functor, owns the per-call context)
  Path A — legacy:  ActivationBase subclasses (ReLU, Sigmoid, …)
                    Each class owns a Tier-1 functor fn_ + an ActivationForward ctx_.
  Path B — new:     ActivationsFnLayer<Fn> / ActivationsFnLayerPtr
                    Generic adapter; preferred for new code and plugin authoring.
```

The `ILayer` tree for `ReLU` (Path A):

```
nnstudio::ILayer
  └ nnstudio::builtin::layers::ActivationBase   ← no-ops for parameters()/build()
      └ nnstudio::builtin::layers::ReLU         ← typeName + delegates to ReLUFn
```

The `ILayer` tree for `ActivationsFnLayer<ReLUFn>` (Path B):

```
nnstudio::ILayer
  └ ActivationsFnLayer<ReLUFn>                  ← owns ReLUFn fn_; stores ctx_
```

Both paths produce an `ILayer` that behaves identically from the caller's perspective. Path B is preferred when you want a reentrant, share-friendly activation (e.g. the same `ReLUFn` functor instance used in multiple graph branches) or when writing a plugin.

`Dense` skips `ActivationBase` entirely — it extends `ILayer` directly because it has learnable parameters and a non-trivial `build()`.

What `ActivationBase` contributes

`ActivationBase` factors out the two methods that are identical for every parameterless activation:

```
// Activations have no learnable parameters
std::vector<Parameter*>       parameters()       override { return {}; }
std::vector<const Parameter*> parameters() const override { return {}; }

// Output shape == input shape — just mark built and pass through
Result<Shape> build(const Shape& inputShape) override {
    markBuilt();
    return Result<Shape>(inputShape);
}
```

It also inherits `ctx_` (an `ActivationForward`) that the concrete class's `forward()` populates and `backward()` consumes.

The `IActivation` interface and `ActivationForward`

```
struct ActivationForward {
    Tensor output;      // the activation's output (always present)
    Tensor ctx;         // saved tensor for backward
    bool   ctxIsInput;  // true → ctx is the input x; false → ctx is the output y
};

class IActivation {
    virtual ActivationForward  forward (const Tensor& x)                          const = 0;
    virtual Result<Tensor>     backward(const Tensor& gradOut,
                                        const ActivationForward& fwd)             const = 0;
    virtual std::string_view   name()    const noexcept = 0;
};
```

`IActivation::forward()` returns `ActivationForward` which bundles the output *and* the saved context in one value. The caller (`ActivationBase` or `ActivationsFnLayer`) stores that struct between the forward and backward calls. The functor itself never touches mutable state — it receives the context back as a parameter to `backward()`.

Two different save strategies

Every activation must save enough for `backward()` to compute the derivative. Which tensor to save depends on the math:

Group	Who	`ctxIsInput`	What is in `ctx`	Why
Save input	`ReLUFn`, `LeakyReLUFn`, `GELUFn`	`true`	the input `x`	Derivative depends on the *sign* of the input
Save output	`SigmoidFn`, `TanhActFn`, `SoftmaxFn`	`false`	the output `y`	Derivative expressed via output is cheaper than recomputing from `x`

Per-activation math

ReLU — `f(x) = max(0, x)`

```
forward:   y = clamp(x, 0, ∞)           ctx = x  (ctxIsInput = true)
backward:  dX = gradOut ⊙ (ctx > 0 ? 1 : 0)
           (the ⊙ mask zeros out gradient for every input that was ≤ 0)
```

`B().clamp()` dispatches to the active backend. The backward mask is a direct float loop — no backend call needed, just compare and write.

LeakyReLU — `f(x) = x if x > 0 else α·x`, default `α = 0.01`

```
forward:   y_i = x_i > 0 ? x_i : α·x_i     ctx = x  (ctxIsInput = true)
backward:  dX_i = gradOut_i · (ctx_i > 0 ? 1 : α)
```

`α` is a constructor parameter (`LeakyReLUFn(0.01f)`). It is stored in `alpha_` and persisted via `config()` . Unlike a `Parameter` , it is **not** learned — it is fixed after construction.

Sigmoid — `f(x) = σ(x) = 1 / (1 + e^{-x})`

```
forward:   y = 1 / (1 + exp(-x))        ctx = y  (ctxIsInput = false)
backward:  dX = gradOut ⊙ ctx·(1-ctx)
           (the elegant identity: dσ/dx = σ(1-σ))
```

The backward pass never looks at the original input — it only needs `ctx` (the saved output). `y·(1-y)` is cheaper to compute than expanding `σ(x)·(1-σ(x))` from `x` .

TanhAct — `f(x) = tanh(x)`

```
forward:   y = tanh(x)                  ctx = y  (ctxIsInput = false)
backward:  dX = gradOut ⊙ (1 - ctx²)
           (identity: d/dx tanh(x) = 1 - tanh²(x))
```

Same pattern as Sigmoid: the derivative in terms of the output is simpler.
Note: the functor is named `TanhFn` (not `Tanh`) to avoid collision with `std::tanh` .

Softmax — `f(x)_i = exp(x_i - max) / Σ_j exp(x_j - max)` (row-wise)

```
forward:   Per row: subtract max, exp, divide by row sum.   ctx = y  (ctxIsInput = false)
           The max subtraction is numerical stability only — mathematically neutral.
backward:  dX_i = ctx_i · (gradOut_i - Σ_j gradOut_j·ctx_j)
           where ctx = lastOutput  (the softmax output vector)
```

Softmax backward is NOT element-wise. Every output `s_i` depends on every input (because of the normalisation denominator), which means `∂s_i / ∂x_j ≠ 0` for `i ≠ j` . The full Jacobian is a dense matrix — the implementation computes it using the dot-product form above, which is `O(C)` per sample rather than `O(C²)` .

GELU — `f(x) = 0.5·x·(1 + tanh(√(2/π)·(x + 0.044715·x³)))` (Hendrycks & Gimpel 2016)

```
forward:   element-wise; ctx = x  (ctxIsInput = true)
backward:  chain rule through the tanh approximation (see ActivationFunctors.cpp for derivation)
```

GELU is the standard activation in Transformer and LLM architectures (GPT, BERT). It is a smooth, probabilistic gate: unlike ReLU it is never exactly zero for negative inputs — it has a small negative region, which helps gradients flow.

How activations sit in the XOR pipeline

The XOR model is four `ILayer` objects in a `ComputeGraph` :

```
Dense   (2→4)   ← nnstudio::builtin::layers::Dense
ReLU            ← nnstudio::builtin::layers::ReLU          (Path A — ActivationBase subclass)
Dense   (4→1)   ← nnstudio::builtin::layers::Dense
Sigmoid         ← nnstudio::builtin::layers::Sigmoid        (Path A — ActivationBase subclass)
```

Equivalently, using the Path B adapter:

```
ActivationsFnLayer<ReLUFn>     relu;    // same ILayer, same math, reentrant functor
ActivationsFnLayer<SigmoidFn>  sigmoid;
```

`ComputeGraph` (and any caller of an `ILayer*`) does not know or care which path was used — they are all `ILayer*` . Forward and backward dispatch through the vtable. An activation layer costs essentially zero in parameters: `parameters()` returns `{}` , `build()` just marks itself built, and the math is a simple element-wise loop or backend call.

3.9 Why activations exist — the non-linearity requirement

The sandwich pattern (Dense → ReLU → Dense → Sigmoid) is not decoration. It is mathematically necessary.

A Dense layer is a linear function: `y = Wx + b`.
The composition of two linear functions is still a linear function:

```
layer1: y = W1·x + b1
layer2: z = W2·y + b2
          = W2·(W1·x + b1) + b2
          = (W2·W1)·x + (W2·b1 + b2)
          = W'·x + b'          ← still just one linear transformation
```

This is not an approximation — it is exact algebra. **No matter how many Dense layers you stack without activations, the entire network is mathematically equivalent to a single Dense layer.** You could always collapse them all into one. All those weights, all that training time, for zero additional representational power.

The consequence for XOR: XOR is **not linearly separable**. You cannot draw a single straight line through the four XOR samples that correctly separates class 0 from class 1. A single Dense layer (= a linear classifier) cannot solve XOR. It is a mathematical impossibility, not an engineering limitation.

What ReLU adds: `max(0, x)` is a *non-linear* function. Composing it with a Dense layer creates a **piecewise linear** function — one slope for the active region, zero for the inactive region. Stack several of these and the network can approximate functions that bend, curve, and fold the input space. The universal approximation theorem (Cybenko 1989, Hornik 1991) states that a network with at least one hidden layer and a non-linear activation can approximate any continuous function to arbitrary accuracy, given enough neurons.

The sandwich in concrete terms for XOR:

```
Input space:  four points — (0,0), (0,1), (1,0), (1,1)
Target:       0,           1,     1,     0

Dense(2→4):   applies 4 different linear cuts through the 2D input space
ReLU:         gates each cut — "is this linear feature active or not?"
Dense(4→1):   combines the 4 gated features into one scalar vote
Sigmoid:      squashes the vote into a probability in (0,1)
```

The ReLU layer is the pivot. Without it, Dense→Dense→Sigmoid is still just one linear classifier — which is proven to fail on XOR. With ReLU in between, the network can carve out the non-convex region that XOR requires.

Why Sigmoid at the end and not ReLU? Sigmoid squashes to (0,1), which matches the probability interpretation of the BCE loss. ReLU output is unbounded — `BCE::compute()` would take `log(p)` on a value that might be 5.0, which is valid math but meaningless as a probability. The activation at the final layer is chosen to match the loss function's expectation, not for non-linearity.

The sandwich in one sentence: Dense layers are the thinkers — the places where learned knowledge lives as weight values after training. Activations are the mandatory non-linear glue that prevents all the thinkers from mathematically merging into one, and gives the whole structure the ability to model problems that aren't straight lines.

3.10 Architecture templates — how far is stacking constrained?

There is **one hard mathematical constraint** and everything else is engineering choice:

> Every pair of consecutive Dense layers must have at least one non-linear layer between them.

That is it. The constraint says nothing about which non-linearity, how many Dense layers, in what width or depth, or whether the graph is even linear. Everything else is a template — a pattern that has been proven useful, not a rule.

The major templates in common use:

Template	Characteristic stacking	What it models
MLP (what XOR is)	Dense → ReLU → Dense → Sigmoid	Tabular data, classification, regression
CNN	Conv2D → ReLU → Pool → Dense	Spatial data — images, audio spectrograms

Template	Characteristic stacking	What it models
RNN / LSTM	recurrent cell looped over sequence steps	Sequential data — text, time series
Transformer	Attention → LayerNorm → FFN (Dense→GELU→Dense)	Long-range dependencies — language models, vision
ResNet	Dense → ReLU → Dense + skip connection back to input	Very deep networks — the skip connection stops vanishing gradients
Autoencoder	Encoder (shrinking Dense stack) → Decoder (growing Dense stack)	Compression, anomaly detection, unsupervised

None of these is the "only" valid architecture. They are solutions that happened to work well enough that the community converged on them. New architectures appear regularly — Mixture of Experts (MoE), State Space Models (Mamba, 2023), Kolmogorov-Arnold Networks (KAN, 2024, replacing Dense layers entirely with learnable spline functions on edges rather than fixed activations on nodes).

What NNStudio's design allows:

- `Sequential` : any ordering of any `ILayer` descendants. You can put Sigmoid before ReLU, repeat Dense ten times, or put a custom plugin layer anywhere. Nothing in the engine enforces a template.
- `ComputeGraph` (Chapter 9 / Phase 3): DAG topology. Skip connections, multi-head branches, encoder-decoder splits — anything that is a directed acyclic graph.
- Plugin SDK (Phase 2): any `ILayer` (or future `IActivation`) implementation, including layer types that don't exist yet.

What NNStudio's design does NOT currently include:

- `Conv2D` — not implemented yet (Phase 1 TODO). The interface (`ILayer`) supports it; the implementation doesn't exist.
- Recurrent / stateful layers — `Sequential::backward()` assumes one forward = one backward. Recurrent layers require unrolling across time steps; this needs a different `ILayer` contract or explicit loop in the training loop. Not blocked, but not designed for yet.
- Attention mechanism — requires `ComputeGraph` (non-sequential topology) plus `MultiHeadAttention` layer implementation. Both are Phase 3+ items.

So: NNStudio is currently a full MLP workbench, with the architecture to become a Transformer workbench once Phase 2/3 are complete. The stacking variability goal is structurally intact — the engine imposes no template. The limitation is which layer *types* are implemented, not how they can be combined.

Chapter 4 — The Ordered Pipeline (`Sequential`)

Files:
`core/include/nnstudio/core/Layer.h` — `ILayer` + `Sequential` class definition
`core/src/layers/Layer.cpp` — `Sequential` implementation

> **Call-chain trace:** `docs/examples/sequential-call-chain.cpp`
> A single annotated file that walks through every virtual call in one forward pass and one backward pass for the XOR model — readable without a debugger.

Overview (plain language)

`Sequential` is the simplest answer to *"how do we wire layers into a network?"*.
It owns an ordered list of `ILayer` instances and threads data through them one by one — left-to-right on `forward()` , right-to-left on `backward()` . The remarkable design choice is that `Sequential` itself **implements** `ILayer` : it has the same `forward()` , `backward()` , `build()` , and `parameters()` as any single layer. This means you can nest a `Sequential` inside another one, or hand it directly to `Trainer` — the Trainer never needs to know whether it received one `Dense` or a deep nested pipeline.

Ownership is via `std::vector<LayerPtr>` (`shared_ptr<ILayer>`). Each `model.add<Dense>(...)` call constructs the layer on the heap and pushes ownership into that vector. The layers live exactly as long as the `Sequential` does.

`build(inputShape)` chains shapes through the layer list: the output shape of layer N becomes the input shape for layer N+1, so each layer allocates its own weight matrices exactly sized for whatever arrives before it. `parameters()` flattens all sub-layer parameters into one `std::vector<Parameter*>` — the single list the Optimizer reads when updating weights.

Sequential *is* an ILayer

`Sequential` does not float above the layer system — it **implements** `ILayer` directly.
This single decision has large consequences:

- You hand a `Sequential` to `Trainer` via an `ILayer&` reference — same as a bare `Dense`.
- You can nest a `Sequential` inside another `Sequential` (a sub-network is itself a layer).
- `ComputeGraph` (the DAG variant) also implements `ILayer` — the Trainer never needs to know which it received.

Ownership: `std::vector<LayerPtr>`

```
private:
    std::vector<LayerPtr> layers_;   // LayerPtr = std::shared_ptr<ILayer>
```

Layer memory is heap-allocated and owned by `Sequential` exclusively. Adding a layer:

```
template<typename L, typename... Args>
Sequential& add(Args&&... args) {
    return add(std::make_shared<L>(std::forward<Args>(args)...));  // construct on heap
}                                                                   // push to layers_

// Returns *this — that is the only reason the fluent chain compiles:
model.add<Dense>(4, true, WeightInit::GlorotUniform)
     .add<ReLU>()
     .add<Dense>(1, true, WeightInit::GlorotUniform)
     .add<Sigmoid>();
```

All four layers are destroyed automatically when `model` goes out of scope.

`build()` — the shape relay

No weight matrices exist until `build()` is called. `Sequential::build(inputShape)` chains shapes through the list:

```
Shape current = inputShape;
for (auto& layer : layers_) {
    auto r = layer->build(current);  // "your input will be shape X"
    current = r.value();             // "my output will be shape Y" → next layer's input
}
```

Output shape of layer N becomes input shape of layer N+1. Each `Dense` inside allocates `W_[outF × inF]` and `b_[outF]` at this point — sized precisely for what arrives before it. Shape mismatch at any step propagates the error immediately; nothing after that failed layer is built.

`forward()` — data flows left to right

```
Tensor current = x;
for (auto& layer : layers_) {
    current = layer->forward(current).value();
}
return current;
```

Each layer transforms the tensor and the result becomes the next layer's input. The individual layers save `lastInput_` and `lastOutput_` quietly during their own `forward()` — `Sequential::forward()` itself stores nothing. The pipeline is stateless at the container level; state accumulates inside the leaves.

`backward()` — the same pipe, reversed

```
Tensor grad = gradOut;
for (auto it = layers_.rbegin(); it != layers_.rend(); ++it) {
    grad = (*it)->backward(grad).value();
}
return grad;
```

`rbegin()` to `rend()` — the gradient seed enters at the last layer and travels back through the list in reverse. Each layer accumulates `dW` / `db` into its own parameters during this pass, then returns `dX` to become the gradient input for the layer before it. `Sequential` is a routing relay only — no accumulation here.

Critical ordering constraint: `backward()` must be called after `forward()` on the same data. Each layer needs its `lastInput_` (saved in `forward()`) to compute `dW = gradOut^T @ lastInput_`. Wrong order → garbage gradients.

`parameters()` — the flat collector

```
for (auto& layer : layers_) {
    auto ps = layer->parameters();
    all.insert(all.end(), ps.begin(), ps.end());
}
```

The Optimizer calls `model.parameters()` once and receives a flat `std::vector<Parameter*>` — direct mutable pointers to every weight and bias in the network, across all layers at every nesting depth. Adam applies the same update rule uniformly to every element of that list. It never needs to know which layer a parameter belongs to.

`setTraining()` — mode propagation

```
for (auto& l : layers_) l->setTraining(t);
```

Layers such as `Dropout` and `BatchNorm` behave differently at train vs eval time. One call to `model.eval()` propagates the flag to every child simultaneously. `model.train()` reverses it before resuming training.

ComputeGraph — the DAG alternative (planned)

`Sequential` covers the linear pipeline: layer N feeds exactly layer N+1.
`ComputeGraph` handles skip connections, residual blocks, and branching architectures (encoder-decoders, multi-head attention) — the general directed acyclic graph. Both implement `ILayer`. For XOR, `Sequential` is all that is needed, and it is sufficient to understand the full forward/backward chain end-to-end before introducing graph complexity.

Chapter 5 — Judging the Output (`Loss`)

Files:
`core/include/nnstudio/core/Losses.h`
`core/src/losses/Losses.cpp`

Overview (plain language)

> *Loss compares prediction vs target → scalar float.*
> *Also produces the seed gradient dLoss/dPrediction.*

That two-liner is the complete contract. Loss is the only component in the system that can see both the network's answer and the correct answer at the same time. From that comparison it does two things: it produces a single scalar that tells you *how wrong* the network was, and it produces a gradient Tensor of the same shape as the prediction that tells `Sequential::backward()` *which direction is wrong.*

`ILoss` is a pure abstract interface with exactly two methods:

```
Result<Tensor> compute (const Tensor& pred, const Tensor& target);  // → scalar Tensor{1}
Result<Tensor> gradient(const Tensor& pred, const Tensor& target);  // → dL/dPred, same shape as pred
```

`compute()` and `gradient()` are **separate calls** — `Trainer` calls them one after the other (steps 3 and 4 of `trainStep()`). There is no internal state between them. Both receive the same `pred` and `target`, so no caching is needed.

The four loss functions

MSE — Mean Squared Error

Used for regression (predicting a continuous value). Penalises large errors more than small ones (quadratic).

$$L = \frac{1}{N}\sum_i (p_i - y_i)^2 \qquad \frac{\partial L}{\partial p_i} = \frac{2(p_i - y_i)}{N}$$

```
// compute():
Tensor diff = sub(pred, target);         // pred - target, elementwise
Tensor sq   = mul(diff, diff);           // square elementwise
return mean(sq, -1, false);             // global mean → scalar

// gradient():
const float invN = 2.0f / pred.numel();
return mulScalar(sub(pred, target), invN);
```

BCE — Binary Cross-Entropy

Used when the output is a single probability (a Sigmoid output). The standard loss for binary classification — XOR uses this.

$$L = -\frac{1}{N}\sum_i \left[y_i \log p_i + (1 - y_i)\log(1 - p_i)\right]$$

$$\frac{\partial L}{\partial p_i} = \frac{1}{N}\left(-\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i}\right)$$

`kEps = 1e-7f` clips `pred` away from exact 0 and 1 before taking `log` — without this, `log(0) = -∞` would produce `NaN` gradients and training would silently collapse.

```
float p = clamp(pred.flat(i), kEps, 1.0f - kEps);  // guard against log(0)
loss += -(y * log(p) + (1-y) * log(1-p));
```

CrossEntropy — Categorical Cross-Entropy

Used when the output is a probability distribution over multiple classes (a Softmax output). Each target value is a probability (one-hot or soft label).

$$L = -\frac{1}{B}\sum_i y_i \log p_i \qquad \frac{\partial L}{\partial p_i} = -\frac{y_i}{p_i \cdot B}$$

Note: normalised by **batch size** (`pred.shape()[0]`), not total element count. This keeps the gradient magnitude independent of how many classes there are.

HuberLoss — Smooth L1

Hybrid: behaves like MSE when the error is small (quadratic, smooth gradient), like MAE when error is large (linear, bounded gradient). Controlled by `delta_` (default 1.0).

$$L_i = \begin{cases} \frac{1}{2}r_i^2 & |r_i| \le \delta \\ \delta(|r_i| - \frac{1}{2}\delta) & |r_i| > \delta \end{cases} \qquad r_i = p_i - y_i$$

The transition at `delta_` prevents the exploding gradients that large outliers cause with pure MSE, while preserving smooth curvature (and therefore well-behaved Adam moments) in the normal range.

Why Loss has no state

Loss functions hold no mutable members between calls (except `HuberLoss::delta_` which is a constructor parameter, not call state). This means the same `ILoss` instance can be shared across threads, called in any order, or used for both training loss and validation metrics simultaneously — without any synchronisation.

Chapter 6 — Fixing the Weights (`Optimizer`)

Files:

`core/include/nnstudio/core/Optimizers.h`

`core/src/optimizers/Optimizers.cpp`

Overview (plain language)

> *Optimizer reads W_.grad_ for every parameter.*
> *Nudges W_ downhill on the loss surface.*

After a backward pass every `Parameter.tensor` has `grad_` populated — a Tensor of the same shape recording `dLoss/dWeight` for every single float. The optimizer's job is to read those numbers and make a small update to `data_[]` that moves the weight in the downhill direction. It then steps aside until the next backward pass.

`IOptimizer` has one method that matters:

```
void step(std::vector<Parameter*>& params);   // reads grad(), updates data_[]
```

And one convenience that delegates to `Tensor::zeroGrad()` :

```
void zeroGrad(std::vector<Parameter*>& params);  // called by Trainer BEFORE forward
```

Both methods work off the same flat `vector<Parameter*>` that `Sequential::parameters()` returns — every weight and bias in the whole model, in layer order, in one list.

SGD with momentum

The simplest update rule: move each weight by `learning_rate × gradient` .
With momentum, a velocity buffer accumulates past gradients so the update has inertia — it resists sharp direction changes and speeds up in consistent directions.

$$v_t = \mu\, v_{t-1} + g_t \qquad \theta_t = \theta_{t-1} - \alpha\, v_t$$

```
// velocity buffer keyed by parameter's data pointer — allocated lazily on first step
vel[i] = momentum_ * vel[i] + g;   // accumulate
p->tensor.flat(i) -= lr_ * vel[i]; // update weight
```

`velocities_` is an `unordered_map<const float*, vector<float>>` , keyed by the raw data pointer of each parameter. The pointer is stable for the lifetime of the `Tensor` object, so this is a safe identity key. The vector is allocated lazily (zero-initialised) on the first `step()` call for each parameter.

Adam — Adaptive Moment Estimation

Adam maintains two moment buffers **per parameter, per weight float**:

- `m` — exponential moving average of the gradient (1st moment: direction)
- `v` — exponential moving average of the squared gradient (2nd moment: variance)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\, g_t^2$$

At the start of training both `m` and `v` are zero, so their estimates are biased low. Bias correction hats remove this:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

The update divides by $\sqrt{\hat{v}} + \varepsilon$ — this makes the effective learning rate **per-weight adaptive**: a weight that has been receiving large, consistent gradients gets a smaller step; a weight that rarely activates gets a proportionally larger step.

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

```
const float bc1 = 1.0f - pow(beta1_, step_);   // bias correction denominators
const float bc2 = 1.0f - pow(beta2_, step_);

mBuf[i] = beta1_ * mBuf[i] + (1.0f - beta1_) * g;
vBuf[i] = beta2_ * vBuf[i] + (1.0f - beta2_) * g * g;

float mHat = mBuf[i] / bc1;
float vHat = vBuf[i] / bc2;
p->tensor.flat(i) -= lr_ * mHat / (sqrt(vHat) + eps_);
```

Default hyperparameters (β_1=0.9 , β_2=0.999 , ε=1e-8 , lr=1e-3) work well across a wide range of problems without tuning — which is why Adam is the default choice for new models.

One neuron, one weight, one step — concrete numbers

The simplest possible Dense: one input `x` , one weight `w` , one bias `b` , one output.
Using the XOR values from the backward-pass trace:

```
Forward:   x=0.8,  w=0.3,  b=0.1  →  y = 0.3×0.8 + 0.1 = 0.34
Seed grad arriving from Sigmoid+BCE:  dX3 = -0.5

Dense::backward computes:
  dW = dX3 × lastInput_ = -0.5 × 0.8 = -0.4   ← stored in w.grad_
  db = dX3              = -0.5               ← stored in b.grad_
  dX = dX3 × w        = -0.5 × 0.3 = -0.15  ← returned to previous layer
```

Adam::step now reads `w.grad_ = -0.4` . Starting from `m=0` , `v=0` (first step):

```
m = 0.9×0  + 0.1×(-0.4) = -0.04
v = 0.999×0 + 0.001×(-0.4)² = 0.00016

Bias correction (step 1 — bc1=0.1, bc2=0.001):
  mHat = -0.04  / 0.1   = -0.4
  vHat =  0.00016 / 0.001 =  0.16

Update:
  w_new = 0.3 − 0.001 × (−0.4) / (√0.16 + 1e-8)
        = 0.3 − 0.001 × (−0.4) / 0.4
        = 0.3 + 0.001
        = 0.301
```

The weight moved from `0.3` to `0.301` . The negative `dW` (-0.4) means "loss decreases if `w` increases" — so Adam nudged it upward. The step size is exactly `lr=0.001` on step 1 because the bias-corrected gradient and its RMS both simplify cleanly.

Step 2 with `dW=-0.3` (weaker signal next sample):

```
m = 0.9×(-0.04) + 0.1×(-0.3) = -0.036 − 0.03 = -0.066
```

The previous gradient is still influencing direction — that is the momentum. If the next sample happened to produce `dW=+0.1` (opposite direction), `m` would still be negative and the update would still go upward, just more weakly. Adam resists reversals.

Standard Adam applies weight decay by adding `λ·θ` to the gradient before computing moments. This couples weight decay to the gradient scale, making its effect inconsistent across parameters.

AdamW applies weight decay **directly to the weight, before the Adam update**, so it is decoupled from the gradient magnitude:

```
p->tensor.flat(i) *= (1.0f - lr_ * decoupledDecay_);   // shrink the weight first
// ... then normal Adam update on the gradient
```

This restores the correct regularisation behaviour and is the recommended default for Transformer-based models.

The `frozen` flag

Every `Parameter` has a `bool frozen` field. Both SGD and Adam check it:

```
if (p->frozen || !p->tensor.hasGrad()) continue;
```

A frozen parameter is never updated. This is the mechanism for transfer learning: load a pretrained model, freeze all layers except the final one, train only the head. No separate parameter list needed — the same `parameters()` flat vector is used, with `frozen = true` on the layers you don't want to change.

LR Schedulers

`ILRScheduler::onStep(opt, globalStep)` is called by the training loop once per epoch (or step). The only concrete implementation is `StepDecayScheduler`: multiply `lr` by `gamma` every `stepSize` steps.

```
if (globalStep % stepSize_ == 0)
    opt.setLR(opt.lr() * gamma_);   // e.g. lr × 0.1 every 100 epochs
```

`CosineAnnealingLR` is defined in the header but not yet implemented (stub in TODO).

Chapter 7 — Running the Loop (`Trainer`)

Files:
`core/include/nnstudio/core/Trainer.h`
`core/src/training/Trainer.cpp`

Overview (plain language)

`Trainer` is the conductor. It owns nothing — meaning it holds **references** (`&`) to a model, a loss function, and an optimizer, but it does not create, copy, or destroy any of them.

The actual objects and their ownership in practice (as seen in the XOR test):

- **`Sequential model`** — a local variable that owns the layers. Each `model.add<Dense>(...)` call creates the `Dense` object on the heap and stores it via `std::shared_ptr<ILayer>` inside `Sequential::layers_`. The layers live exactly as long as `model` does.
- **`BCE bce`** and **`Adam adam`** — further local variables in the same scope.
- **`Trainer trainer(model, bce, adam)`** — receives `ILayer&`, `ILoss&`, `IOptimizer&`. When `trainer` goes out of scope it drops references only; the real objects are unaffected.

This separation is intentional: you can train the same model with two different optimizers, swap the loss function, or run evaluation after training — all without moving or copying the model. The scope of "I want a model" and the scope of "I want to train it right now" are legitimately different.

One counter-intuitive subtlety: the `Trainer` does not touch weights directly — it only calls `optimizer.step(params)`. The optimizer owns the update arithmetic. The layer owns the parameters. The trainer only orchestrates the sequence.

`trainStep()` — the six-step atomic unit of learning

`trainStep()` is the atomic unit of learning — one forward + backward + weight update.
Everything else (`trainEpoch`, `train`) is a counted loop calling it.

```
1. zeroGrad        — clear all grad_ fields from the previous step
2. forward         — X flows through all layers; each saves lastInput_
3. loss.compute    — compare prediction to target → scalar float L
4. loss.gradient   — dL/dPrediction — the seed gradient
5. backward        — gradient flows in reverse: last layer → first layer
                     each layer accumulates dW/db into grad_, returns dX downward
6. optimizer.step  — Adam/SGD reads grad_ from every Parameter, updates data_
```

Adam update rule (Step 6)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\, g_t^2$$

$$\theta_t = \theta_{t-1} - \alpha \, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

`m` = running mean of gradients (momentum).
`v` = running mean of squared gradients (adaptive per-weight learning rate).
Bias-correction hats (^) stop the estimates from being near-zero at the start of training.

> **Implementation:** `core/src/optimizers/Optimizers.cpp`

Chapter 8 — The XOR Milestone

File: `tests/core/test_trainer_xor.cpp`

XOR is the classic "hello world" for neural networks:
the simplest problem that is *not* linearly separable — no single line divides the 0s from the 1s.
It proves the engine works end-to-end: forward, backward, optimizer, convergence.

```
[0,0]→0   [0,1]→1   [1,0]→1   [1,1]→0
```

Architecture: `Dense(2→4) → ReLU → Dense(4→1) → Sigmoid`

Layer	Purpose
`Dense(2→4)`	Projects 2D input into 4D space where XOR *becomes* linearly separable
`ReLU`	Non-linearity — without it, two Dense layers collapse into one linear function
`Dense(4→1)`	Collapses 4D features into one output logit
`Sigmoid`	Squashes logit to (0,1) — interpretable as probability

Test passes when: `finalLoss < 0.05` *and* all 4 predictions round correctly.
Runs in ~69ms for 3000 epochs on 4 samples (~170k weight updates/second, single CPU thread).

Chapter 9 — Step-by-Step Visualization (Planned — Phase 3 UI)

> This chapter describes a **planned feature**, not yet implemented.

The problem

A beginner looking at the network needs to see:

- the actual float values arriving at each neuron
- what the weighted sum produces before and after the activation
- how the gradient signal changes as it flows backwards

None of this is visible from the outside once training is running at full speed.

The solution: `EvalTrace`

We will add an opt-in **trace mode** to the engine.
When active, `forward()` records every layer's input and output into an `EvalTrace` object instead of discarding them:

```
// Planned API (not yet implemented — see TODO.md Phase 3)
struct LayerTrace {
    std::string   layerName;
    Tensor        input;    // what arrived
    Tensor        output;   // what was produced
    Tensor        gradIn;   // gradient that arrived in backward (optional)
    Tensor        gradOut;  // gradient that was produced
};

struct EvalTrace {
    std::vector<LayerTrace> layers;   // one entry per layer, in forward order
};
```

The `Trainer` (or a new `TracingForward` wrapper) populates this when the UI is in "explain" mode. Normal training ignores it entirely — zero cost.

What the UI will show

For each layer in the graph (Phase 3 Qt UI):

1. **Neuron view** — a grid of circles. Each circle = one neuron.
   Colour/size = activation value. Click a neuron to see its individual `w·x+b` breakdown.
2. **Weight heatmap** — the weight matrix `W` as a colour grid.
   After each step, cells that changed significantly flash.
3. **Gradient flow** — arrows between layers show `dX` magnitude.
   Thin arrow = small gradient (vanishing gradient problem visible here).
4. **Step-by-step mode** — pause after each of the 6 `trainStep` phases,
   inspect the state at that exact moment.

> **Engine hook needed before UI:** `ILayer::forward()` needs an optional `EvalTrace*` parameter (defaulting to `nullptr` = no tracing).
> This is a small, non-breaking addition — all existing code passes `nullptr` implicitly.

Chapter 10 — Phase 1 + 2 Complete: The Full Toolkit

> **What changed since Chapters 1–8?**
> Chapters 1–8 documented the original spine: `Tensor → IBackend → Dense → Sequential → Loss → Optimizer → Trainer → XOR`.
> This chapter covers everything built on top of that spine during Phase 1 and Phase 2 before any UI work begins.
> All components listed here are **fully implemented, tested, and passing** as of commit `055eb99` (170/170 tests, 1.58 s).

§10.1 — The Activation Paradigm Shift (ADR-020)

The original design — `ActivationBase`

The first activation layers (`ReLU`, `Sigmoid`, `TanhAct`, `LeakyReLU`, `Softmax`, `GELU`) were written as concrete `ILayer` subclasses that all inherit `ActivationBase`. `ActivationBase` saves `lastInput_` or `lastOutput_` as a mutable member field between `forward()` and `backward()`.

That works fine for single-threaded sequential training. It breaks under:

- **Multi-threaded batch evaluation** — thread A's forward clobbers thread B's saved context before thread B calls backward.
- **ComputeGraph replay** — the graph can re-run `forward()` without calling `backward()`, leaving stale context.
- **Quantised / fused kernels** — the saved context may not be a plain `Tensor` at all.

ADR-020 — `IActivation` (stateless functor)

`IActivation` (`include/core/IActivation.h`) is a pure-virtual interface for **stateless** activation functions:

```
class IActivation {
public:
    // Returns output + saved ctx — NO mutable state on the object.
    virtual ActivationForward forward(const Tensor& x) const = 0;

    // Receives the ActivationForward from the matching forward() call.
    virtual Result<Tensor> backward(const Tensor& gradOut,
                                    const ActivationForward& fwd) const = 0;
    virtual std::string_view name() const noexcept = 0;
};
```

`ActivationForward` is a plain struct: `{ Tensor output; Tensor ctx; bool ctxIsInput; }` .
`ctxIsInput = true` → ctx holds the **input** x (ReLU, LeakyReLU, GELU — need x's sign/value).
`ctxIsInput = false` → ctx holds the **output** y (Sigmoid, Tanh, Softmax — derivative is cheaper via output).

Because the functor has no mutable members, **one instance can be shared across threads and graph nodes**.

`ActivationFunctors.h` — the six stateless functors

Functor	`ctxIsInput`	formula
`ReLUFn`	true	$f(x) = \max(0, x)$
`LeakyReLUFn(alpha)`	true	$f(x) = x$ if $x > 0$ else αx
`SigmoidFn`	false	$f(x) = 1/(1 + e^{-x})$
`TanhActFn`	false	$f(x) = \tanh(x)$
`SoftmaxFn`	false	row-wise numerically stable softmax
`GELUFn`	true	$0.5x(1 + \tanh(\sqrt{2/\pi}\,(x + 0.044715\,x^3)))$

All six functors implement `IActivation` as zero-mutable-state structs. The legacy `ActivationBase` layer subclasses delegate their `forward()` / `backward()` to these functors, eliminating the non-reentrant mutable fields while keeping backward compatibility with existing tests.

`ActivationsFnLayer<Fn>` — the adapter

`ActivationsFnLayer<Fn>` (`include/builtin/layers/ActivationsFnLayer.h`) wraps any `IActivation` implementor into an `ILayer` that can be dropped into any `Sequential` . The **layer** (not the functor) owns the per-call `ActivationForward` context; the functor stays stateless.

```
// Option A — owning (functor lives inside the layer)
auto layer = ActivationsFnLayer<ReLUFn>{};
auto layer = ActivationsFnLayer<LeakyReLUFn>{ LeakyReLUFn{0.2f} };

// Option B — non-owning (functor shared externally)
auto shared = std::make_shared<GELUFn>();
auto layer  = ActivationsFnLayerPtr{ shared.get() };
```

New code should use `ActivationsFnLayer` . Legacy code using `ReLU : ActivationBase` continues to work unchanged.

§10.2 — The Full Layer Suite

All layers below are in `include/builtin/layers/` and fully implement `ILayer::build() / forward() / backward()` .

Dropout

```
Input  [N, F] → random zero mask (training) or identity (eval) → Output [N, F]
```

- **Inverted dropout**: surviving elements are scaled by `1/(1 - dropRate)` so the expected activation magnitude is unchanged between training and eval.
- `setSeed(uint32_t)` for reproducible tests.
- `setTraining(bool)` : eval mode makes `forward()` the identity; backward is transparent.

BatchNorm1d

```
Input [N, F]  →  normalize over N  →  gamma[F] * x_hat + beta[F]  →  Output [N, F]
```

- Tracks `running_mean[F]` and `running_var[F]` during training (exponential moving average, momentum configurable).
- Eval mode uses the running statistics.
- Full backward through the normalisation including `dW(gamma)` , `db(beta)` , and `dX` through the shared mean/variance.
- Learnable parameters: `gamma[F]` (scale, init = 1), `beta[F]` (shift, init = 0).

LayerNorm

```
Input [N, D]  →  normalize over D (per sample)  →  gamma[D] * x_hat + beta[D]  →  Output [N, D]
```

- Normalises over the **feature dimension** (last dim), unlike BatchNorm which normalises over N.
- No running statistics — LayerNorm is stateless outside the learnable `gamma` / `beta` .
- Efficient backward via saved `x_hat` and `rstd` (reciprocal std-dev).
- `normalizedDim` inferred from input shape when passed as `-1` .

Embedding

```
Input  [N, seqLen]  (float-stored integer IDs)
  →  row lookup in weight[vocabSize, embDim]
  →  Output [N, seqLen, embDim]
```

- The weight matrix is a `[vocabSize × embDim]` lookup table initialised with uniform random values.
- Forward: for each token ID `t` , copy `weight[t, :]` to the output row.
- Backward: accumulates `gradOut[b, s, :]` into `dW[t, :]` for every `(b, s)` position that selected token `t` . Multiple positions selecting the same token accumulate.
- Integer token IDs are stored as `float32` and cast to `int64_t` internally (Phase 1 limitation; a dedicated `Int32/Int64 DType` path is Phase 4).

MultiHeadAttention

```
Input [N, L, d_model]
  →  Wq, Wk, Wv projections  →  Q, K, V [N, L, d_model]
  →  split h heads           →  [N*h, L, d_k]
  →  scaled dot-product      →  Attn [N*h, L, L]  (softmax(QK^T / √d_k))
  →  Attn @ V                →  [N*h, L, d_k]
  →  merge heads + Wo proj   →  Output [N, L, d_model]
```

- 8 learnable parameters: `Wq, Wk, Wv, Wo` (each `[d_model, d_model]`) and matching biases `bq, bk, bv, bo` (each `[d_model]`).
- `causal=true` : masks score matrix above the diagonal to `-∞` before softmax (autoregressive generation).
- `dropout` argument accepted; application at attention weights is Phase 4.
- Full backward through all four projections.

Conv2D

```
Input [N, C_in, H, W]
  →  kernel[C_out, C_in, kH, kW] slide with stride / padding
  →  Output [N, C_out, outH, outW]
```

- NCHW memory layout throughout.
- `outH = (H + 2*padding − kH) / stride + 1` ; same for W.
- Phase 1 implementation: direct nested-loop convolution. No `im2col` ; correctness over peak throughput. Backend GEMM acceleration is Phase 4.
- Optional bias `[C_out]` .
- Full backward: `dInput` , `dKernel` , `dBias` .

§10.3 — The Full Loss Suite

All losses are in `include/builtin/losses/Losses.h`. All are stateless: `compute()` and `gradient()` are separate pure methods on the same `(pred, target)` pair. The `kEps` clamp `[1e-7, 1-1e-7]` on logarithm inputs guards against `log(0) = -∞ → NaN` (the #1 silent training failure cause).

Class	Formula	Use case
`MSE`	$\frac{1}{N}\sum(p_i - t_i)^2$	Regression; delegates to `IBackend::mse`
`BCE`	$-\frac{1}{N}\sum[t\log p + (1-t)\log(1-p)]$	Binary classification (sigmoid output required)
`CrossEntropy`	$-\frac{1}{N}\sum\sum t_{ij}\log p_{ij}$	Multi-class (softmax output required)
`HuberLoss(δ)`	$0.5(p-t)^2$ if $\|p-t\| \le \delta$; else $\delta(\|p-t\| - \frac{\delta}{2})$	Robust regression; less penalty for outliers

§10.4 — The Full Optimizer Suite

All optimizers are in `include/builtin/optimizers/Optimizers.h`. All implement `IOptimizer::step(params)` and respect `Parameter::frozen`. Moment buffers are keyed on raw `Parameter*` address (stable for the model lifetime).

Class	Hyperparameters	Notes
`SGD`	`lr, momentum, weightDecay`	Velocity buffer per parameter when `momentum > 0`
`Adam`	`lr, β1=0.9, β2=0.999, ε=1e-8, weightDecay`	Bias-corrected 1st/2nd moments; serialisable state for checkpoint resume
`AdamW`	`lr, β1, β2, ε, decoupledDecay=0.01`	Weight decay applied directly (`θ *= 1-λ`) before Adam update — decoupled from adaptive lr
`RMSProp`	`lr, α=0.99, ε=1e-8, weightDecay`	EMA of squared gradients: $E[g^2]_t = \alpha E[g^2]_{t-1} + (1-\alpha)g^2$

`StepDecayScheduler(stepSize, gamma)` — implements `ILRScheduler::onStep()`: multiplies `optimizer.lr` by `gamma` every `stepSize` global steps.

§10.5 — Checkpoint System

`Checkpoint` (`include/core/Checkpoint.h`) provides stateless `save()` / `load()` for binary `.nns` checkpoint files.

File format (all integers little-endian):

```
"NNSC" magic + uint16 version=1
"WS" section  — count × (name_len, name, NNS1 tensor)
"OS" section  — optimizer type string + step counter t +
                count × (has_state flag + m tensor + v tensor)
"TC" section  — uint64 globalStep + uint64 epoch
"EN" end tag
```

Key design choices:

- Raw gradients are **not** saved — they are recomputed on the first backward pass after resume.
- `load()` is lenient: unknown section tags are skip-forwarded (forward-compat with future sections).
- Optimizer state (`m`, `v`, `t`) is indexed by parameter **position** in `ILayer::parameters()`, not by name — name collisions are tolerated.
- `Checkpoint::save()/load()` are called by `Trainer` automatically when `saveEvery > 0` is configured.

§10.6 — `torch_compat` — The Code IS the Model

`include/torch_compat.h` is a **header-only** shim that makes any translation unit that uses only the standard PyTorch / LibTorch C++ API work against the NNStudio engine with a one-line change:

```cpp
// swap exactly this one line:
#include <torch_compat.h>    // ← NNStudio
// #include <torch/torch.h>  // ← LibTorch

auto model = torch::nn::Sequential(
    torch::nn::Linear(4, 8),
    torch::nn::ReLU(),
    torch::nn::Linear(8, 1),
    torch::nn::Sigmoid()
);
auto opt = torch::optim::Adam(1e-3f);
```

The shim maps:

PyTorch name	NNStudio type
`torch::Tensor`	`nnstudio::core::Tensor`
`torch::nn::Linear`	`Dense` (wrapper storing `in_features`)
`torch::nn::Conv2d`	`Conv2D` (wrapper storing `in_channels`)
`torch::nn::Embedding`	`Embedding` (direct alias)
`torch::nn::MultiheadAttention`	`MultiHeadAttention` (arg-order adapter)
`torch::nn::BatchNorm1d`	`BatchNorm1d` (direct alias)
`torch::nn::LayerNorm`	`LayerNorm` (direct alias)
`torch::nn::Dropout`	`Dropout` (direct alias)
`torch::nn::ReLU/Sigmoid/Tanh/GELU/Softmax/LeakyReLU`	Corresponding `ActivationBase` subclasses
`torch::nn::Sequential`	**inline template class** — owns a `vector<unique_ptr<ILayer>>`
`torch::nn::functional::relu/sigmoid/tanh/gelu/softmax/dropout`	Stateless calls on `IActivation` functors
`torch::optim::SGD/Adam/AdamW/RMSprop`	Corresponding optimizer classes
`torch::kFloat32, kFloat16, kInt8, kInt32, kBool`	`DType::*` constants
`torch::kCPU, torch::kCUDA`	`Device::*` constants

`torch::nn::Sequential` in this header is **not** the same as `nnstudio::builtin::layers::Sequential` — it is a self-contained implementation that builds the wire-up at construction time from variadic template arguments, matching PyTorch's interface exactly:

```cpp
auto m = torch::nn::Sequential(
    torch::nn::Linear(4, 8), torch::nn::ReLU(), torch::nn::Linear(8, 1));
m.build({1, 4});
auto y = m.forward(x).value();
```

This shim is the foundation for ADR-030 option (b) — the argument that NNStudio model files should use torch-compat code rather than a custom DSL. The shim demonstrates that torch-compat code is already a first-class citizen in the engine.

The Python-layer equivalent is `python-bridge/nnstudio/torch_compat.py`:

```python
import nnstudio.torch_compat as torch   # instead of: import torch
import nnstudio.torch_compat.nn as nn

model = nn.Sequential()
model.add(nn.Linear(4, 8))
model.add(nn.ReLU())
```

§10.7 — Plugin SDK

The C ABI (`nnstudio_plugin.h`)

The plugin interface is **pure C** — no C++ types cross the ABI boundary. This allows plugins written in any language that can export a C symbol. Every plugin shared library must export exactly one symbol:

```
const NNPluginDescriptor* nnstudio_plugin_descriptor(void);
```

`NNPluginDescriptor` carries: `api_version` , `type` , `id` (reverse-domain string), `name` , `version` , `author` , `license` , `create()` , `destroy()` , and a `vtable*` pointer cast to the type-specific vtable struct.

Defined plugin types (Phase 2 ABI):

`NNPluginType`	vtable struct	Purpose
`NN_PLUGIN_LAYER`	`NNLayerVTable`	Custom ILayer implementor
`NN_PLUGIN_ACTIVATION`	`NNActivationVTable`	Custom activation function
`NN_PLUGIN_OPTIMIZER`	`NNOptimizerVTable`	Custom optimiser
`NN_PLUGIN_TOKENIZER`	`NNTokenizerVTable`	encode / decode / vocab introspection
`NN_PLUGIN_BACKEND`	*(planned Phase 3)*	Custom compute backend
`NN_PLUGIN_INPUT_ADAPTER`	*(planned Phase 3)*	Data reader / pre-processing
`NN_PLUGIN_OUTPUT_ADAPTER`	*(planned Phase 3)*	Post-processing / sink
`NN_PLUGIN_CONTEXT_SOURCE`	`NNContextSourceVTable`	RAG retrieval over query embeddings
`NN_PLUGIN_RUNNER_CLIENT`	`NNRunnerClientVTable`	Remote inference server connector
`NN_PLUGIN_UI_PANEL`	`NNUIPanelVTable`	QML component URL for dockable panels
`NN_PLUGIN_TRUST_UPDATE` (99)	*(TrustUpdateHandler)*	Signed trust-store update packet

The `NNTensorView` struct (read-only window into engine-owned memory: `data, shape, ndim, dtype`) and `NNMutableTensorView` are the only tensor types that cross the ABI — no `std::shared_ptr` , no RTTI, no vtable.

`PluginLoader` — loading sequence

`PluginLoader` (`include/plugin-api/PluginLoader.h`) loads a shared library through a six-step sequence:

1. `TrustVerifier::verify(path)` — check plugin signature against `TrustStore`
2. If trust level < policy minimum → **reject** (never call `dlopen`)
3. `dlopen` (Linux/macOS) / `LoadLibraryW` (Windows)
4. Resolve `nnstudio_plugin_descriptor` symbol
5. Check `api_version == NNSTUDIO_PLUGIN_API_VERSION`
6. Return `LoadedPlugin` (RAII handle; destructor calls `destroy() + dlclose`)

`LoadPolicy` is an enum: `RequireEnterprise` , `RequireCommunity` , `AllowUntrusted` .

Phase 5 plans a **sandboxed** policy that routes through an `nnstudio-runner` sidecar process over gRPC instead of `dlopen` .

Trust Architecture

`TrustStore` (`include/plugin-api/trust/TrustStore.h`) — **trust level hierarchy**:

Level	Description
3 — Root	Embedded in binary; hardware-verified at build time (HSM in production)
2 — Enterprise CA	Issued by Root; customer deploys to employees
1 — Community CA	Issued by Root; open-source plugin authors

Level	Description
0 — Untrusted	No valid signature found

The `.nnts` store file (binary, little-endian) contains a list of `CaEntry` records (DER-encoded X.509 certificates) signed by the Root CA. The Root CA certificate is **not** stored in the file — it is embedded in the binary.

`TrustVerifier` (`include/plugin-api/trust/TrustVerifier.h`) walks the cert chain from the plugin's embedded certificate up to a `CaEntry` in the `TrustStore` and returns a `VerifyResult { TrustLevel level; std::string subjectDN; }` .

`TrustUpdateHandler` (`include/plugin-api/trust/TrustUpdateHandler.h`) verifies a Trust Update Packet (TUP) before calling `TrustStore::addCa()` / `revokeCa()` . No CA can be added or removed except through a signed TUP — this is the anti-supply-chain-attack guarantee.

Plugin signing tool — `src/plugin-api/tools/sign/main.cpp` : CLI tool `nnstudio-sign` that signs a plugin shared library with a Developer CA certificate and embeds the signature as a section in the binary. Used during plugin release pipeline.

§10.8 — Built-in Reference Plugins

BPE Tokenizer (`plugins/bpe_tokenizer/`)

A minimal **byte-level BPE tokenizer** implemented as `NN_PLUGIN_TOKENIZER` . No external vocabulary file needed.

- **Vocabulary size: 319 tokens**
  - IDs 0–255: the 256 raw byte values (byte-fallback, handles any UTF-8 input)
  - IDs 256–318: 63 BPE merge rules — GPT-2-style space+letter merges (`t` , `a` , …) followed by common English bigrams (`th` , `he` , `in` , `er` , …) and trigrams (`the` , `ing` , `the` , …)
- Implements the full `NNTokenizerVTable` : `encode` , `decode` , `free_result` , `vocab_size` , `token_to_str` , `str_to_token`
- `encode()` applies merge rules by rank (lowest rank = highest priority); `decode()` concatenates token strings
- `token_to_str(0)` returns a 1-byte null-byte string (special case: `strlen == 0` , not null ptr)
- Intent: demonstrate the plugin ABI works end-to-end; not for production NLP

Example Activation (`plugins/example_activation/`)

A reference `NN_PLUGIN_ACTIVATION` that implements the **Swish** activation:

$$\text{Swish}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

Derivative: $\sigma(x) + x \cdot \sigma(x)(1 - \sigma(x))$

Demonstrates: owning a per-instance state struct, `NNActivationVTable` wiring, `create()` / `destroy()` lifecycle, `doc_ref()` returning a KB anchor.

Both plugins ship with a `plugin.manifest.json` (JSON5, fields: `id` , `name` , `version` , `author` , `license` , `type` , `description` , `requires_api_version`).

§10.9 — Python Bridge

The Python bridge (`nnstudio/python-bridge/`) exposes the C++ engine to Python via **pybind11** and layers three compatibility façades on top:

pybind11 bindings (`bindings/module.cpp`)

The compiled extension module (`nnstudio.<arch>.so` / `.pyd`) exposes:

- `Tensor` , `DType` , `Device` Python classes
- `zeros` , `ones` , `full` factory functions
- The full `nn` , `optim` , and loss namespaces
- `__version__` string

On Windows (MinGW) the `__init__.py` probes well-known MSYS2 locations and calls `os.add_dll_directory()` so Python ≥ 3.8 can resolve MinGW runtime DLLs at import time.

`nnstudio` package — torch-style import

```
import nnstudio as torch         # drop-in swap
import nnstudio.nn as nn
import nnstudio.nn.functional as F
import nnstudio.optim as optim
```

`nnstudio.torch_compat` module re-exports the same surface with explicit `torch.*` naming so existing scripts can alias the import with zero other changes:

```
import nnstudio.torch_compat as torch
model = torch.nn.Sequential()
```

Keras façade (`nnstudio.keras`)

```
from nnstudio.keras import Model, layers, losses, optimizers, callbacks

seq = layers.Sequential()
seq.add(layers.Dense(4, 1))
m = Model(seq)
m.compile(optimizer=optimizers.Adam(), loss=losses.MeanSquaredError())
history = m.fit(x, y, epochs=10)
```

Sub-modules: `layers` (Dense, Conv2D, Embedding, MHA, BN, LN, Dropout, activations), `losses` (MSE, BCE, Categorical CE, Huber), `optimizers` (Adam, AdamW, SGD, RMSProp), `callbacks` (EarlyStopping, ModelCheckpoint, LRScheduler, CSVLogger, TensorBoard stub).

Runner clients (`nnstudio.runners`)

Five connector classes unified under the `RunnerClient` protocol:

Class	Backend	Protocol
`TritonRunnerClient`	NVIDIA Triton	gRPC + REST (requires `tritonclient`)
`TfServingRunnerClient`	TensorFlow Serving	REST + gRPC
`KServeRunnerClient`	KServe V2	HTTP Inference Protocol
`OnnxRuntimeRunnerClient`	ONNX Runtime	in-process (no network)
`OpenAIRunnerClient`	OpenAI-compatible REST	Ollama, LM Studio, vLLM (requires `httpx`)

All share: `connect(url)` , `load_model(name)` , `infer(model, input)` , `health()` , `disconnect()` plus typed exceptions (`RunnerConnectionError` , `RunnerInferenceError` , `RunnerModelNotFoundError`).

§10.10 — Architecture Templates: Current Implementation Status

The tables below supersede the "Status in NNStudio" entries in the Architecture Templates appendix.

Template 4 — Transformer block (as of Phase 2 / commit `055eb99`)

Layer	Status	Notes
`Embedding`	✅ implemented	`[vocabSize, embDim]` lookup; float-stored IDs (Phase 4: int DType)
`LayerNorm`	✅ implemented	normalises over last dim; learnable γ, β
`GELU`	✅ implemented	as `GELUFn` (`IActivation`) and `GELU : ActivationBase`
`MultiHeadAttention`	✅ implemented	causal mask, 4 weight pairs, full backward
`ComputeGraph` (skip connections)	❌ Phase 3	`Sequential` is still strictly linear
FFN (Dense→GELU→Dense)	✅	`Dense` + `GELU` both ready; wire together in `Sequential`

Residual connections (Template 5) still require `ComputeGraph`. All other individual layer building blocks are available now.

§10.11 — Test Coverage Map (Phase 2 complete)

170/170 tests pass in 1.58 s (`ctest --preset engine-ninja`, RelWithDebInfo, MinGW GCC).

Test file	Count	What it verifies
`tests/builtin/test_layers.cpp`	-	`Dense` forward/backward, shape relay
`tests/builtin/test_activations.cpp`	-	All 6 activation ILayer subclasses + ADR-020 functors
`tests/builtin/test_conv2d.cpp`	-	`Conv2D` padding/stride/backward
`tests/builtin/test_embedding.cpp`	-	`Embedding` lookup + grad accumulation
`tests/builtin/test_losses.cpp`	-	MSE, BCE, CrossEntropy, Huber compute + gradient
`tests/builtin/test_optimizers.cpp`	-	SGD, Adam, AdamW, RMSProp step; frozen params
`tests/core/test_tensor.cpp`	-	Shape/stride/reshape/transpose/operators/serialise
`tests/core/test_compute_graph.cpp`	-	DAG node recording + replay
`tests/core/test_checkpoint.cpp`	-	Save/load weights + Adam state + counters
`tests/core/test_compat.cpp`	-	`CompatibilityChecker` ONNX op-set
`tests/core/test_early_stopping.cpp`	-	Patience counter, delta threshold, restore-best
`tests/core/test_trainer_xor.cpp`	-	Full XOR training loop, 300 epochs convergence
`tests/compat/test_torch_compat.cpp`	-	`torch_compat.h` shim — Sequential, Linear, Adam
`tests/plugin-api/test_plugin_loader.cpp`	-	Trust-gated `dlopen`, AllowUntrusted path
`tests/plugin-api/test_trust_store.cpp`	-	`.nnts` save/load, cert chain
`tests/plugin-api/test_trust_verifier.cpp`	-	Signature verify, TrustLevel resolution
`tests/plugin-api/test_bpe_tokenizer.cpp`	**17**	vocab_size=319, encode/decode round-trip, all 319 token_to_str, str_to_token

> **The engine is now fully equipped to build any standard architecture from scratch.**
> The next step is Phase 3: the `NN_ENABLE_APP=ON` CMake preset, tree-sitter integration for the code parser, and the two-way WYSIWYG model editor (ADRs 030–033).

Appendix A — File Map

Full annotated source tree. Chapter and section tags match the body of this document. The compact chapter-by-chapter navigator appears at the end of the intro (just before Chapter 1) for quick lookup.

```
nnstudio/
│
├── include/                          ← all public headers (no .cpp here)
│   │
│   ├── core/                         ← stable plugin-facing API
│   │   ├── Tensor.h                  Ch.1   data model: shape, strides, grad
│   │   ├── IBackend.h                Ch.2   compute vtable (§D.1–§D.11)
│   │   ├── BackendRegistry.h         Ch.2   singleton + backend() free fn
│   │   ├── Layer.h                   Ch.3,4 ILayer lifecycle + Sequential
│   │   ├── IActivation.h             Ch.3   stateless activation functor contract
│   │   ├── ILoss.h                   Ch.5   loss function contract
│   │   ├── IOptimizer.h              Ch.6   optimizer contract
│   │   ├── Trainer.h                 Ch.7   training loop API
│   │   ├── Result.h                  §1.x   Result<T> error-or-value type
│   │   ├── Device.h                  §1.x   CPU | CUDA | QUANTUM enum
│   │   ├── DType.h                   §1.x   float32 | float16 | int8 enum
│   │   ├── ComputeGraph.h            Ch.4   DAG autograd (Phase 3)
│   │   ├── Checkpoint.h              §10.5  save/load weights + metadata
│   │   ├── EarlyStopping.h           §10.5  patience-based stopping
│   │   ├── FeatureFlags.h            §10.6  FREE | PRO | ENTERPRISE tier gating
│   │   └── CompatibilityChecker.h    §10.6  standard_torch vs nnstudio_extension
│   │
│   ├── builtin/                      ← NNStudio's own implementations
│   │   ├── activations/
│   │   │   ├── Activations.h         Ch.3 §10.1  layer types (ReLU, Sigmoid, …)
│   │   │   ├── Functors.h            Ch.3 §10.1  IActivation stateless functors
│   │   │   └── FnLayer.h             Ch.3 §10.1  ActivationFnLayer<Fn> adapter
│   │   ├── backends/
│   │   │   ├── CpuBackend.h          Ch.2   Eigen SGEMM, row/col-major trick
│   │   │   ├── CudaBackend.h         §D.10  Phase 4 — cuBLAS/cuDNN plan
│   │   │   └── QuantumBackend.h      §D.10  Phase 6 — stub, all methods trap
│   │   ├── layers/
│   │   │   ├── Dense.h               Ch.3   fully-connected layer API
│   │   │   ├── NormLayers.h          §10.2  BatchNorm1d + LayerNorm
│   │   │   ├── Embedding.h           §10.2  token → vector lookup
│   │   │   ├── MultiHeadAttention.h  §10.2  scaled dot-product + heads
│   │   │   └── Conv2D.h              §10.2  2-D convolution (loop impl Phase 1)
│   │   ├── losses/
│   │   │   └── Losses.h              Ch.5 §10.3  MSE, BCE, CrossEntropy, Huber
│   │   └── optimizers/
│   │       └── Optimizers.h          Ch.6 §10.4  SGD, Adam, AdamW
│   │
│   ├── plugin-api/                   ← SDK headers (also in plugin-api/ below)
│   │   ├── nnstudio_plugin.h         §10.7  C ABI: vtable structs, entry points
│   │   ├── PluginLoader.h            §10.7  load / verify / register .nnsp
│   │   └── trust/
│   │       ├── TrustStore.h          §10.7  certificate pin store
│   │       ├── TrustVerifier.h       §10.7  signature verification
│   │       └── TrustUpdateHandler.h  §10.7  TUP: trust update protocol
│   │
│   └── torch_compat.h                §10.6  API shims: Linear→Dense, etc.
│
├── src/                              ← all implementation (.cpp)
│   ├── core/
│   │   ├── Tensor.cpp                Ch.1
│   │   ├── Layer.cpp                 Ch.3,4  ILayer lifecycle + Sequential impl
│   │   ├── Trainer.cpp               Ch.7    trainStep — the 6-step loop
│   │   ├── BackendRegistry.cpp       Ch.2
│   │   ├── ComputeGraph.cpp          Ch.4
│   │   ├── Checkpoint.cpp            §10.5
│   │   └── CompatibilityChecker.cpp  §10.6
│   ├── builtin/
│   │   ├── activations/
│   │   │   ├── Activations.cpp       Ch.3 §10.1  forward/backward per activation
│   │   │   └── Functors.cpp          Ch.3 §10.1  IActivation functor bodies
```

```
│   │   ├── backends/
│   │   │   ├── CpuBackend.cpp          Ch.2   Eigen matmul + all vtable impls
│   │   │   ├── CudaBackend.cpp         Phase 4 stub
│   │   │   └── QuantumBackend.cpp      Phase 6 stub
│   │   ├── layers/
│   │   │   ├── Dense.cpp               Ch.3   forward/backward (annotated)
│   │   │   ├── NormLayers.cpp          §10.2
│   │   │   ├── Embedding.cpp           §10.2
│   │   │   ├── MultiHeadAttention.cpp  §10.2
│   │   │   └── Conv2D.cpp             §10.2
│   │   ├── losses/
│   │   │   └── Losses.cpp              Ch.5
│   │   └── optimizers/
│   │       └── Optimizers.cpp          Ch.6   SGD momentum + Adam update rule
│   └── plugin-api/
│       ├── PluginLoader.cpp            §10.7
│       ├── tools/sign/main.cpp         §10.7  nnstudio-sign CLI tool
│       └── trust/
│           ├── TrustStore.cpp
│           ├── TrustVerifier.cpp
│           └── TrustUpdateHandler.cpp
│
├── plugin-api/                         ← Distributable SDK (not part of engine lib)
│   ├── nnstudio_plugin.h               §10.7  canonical C ABI header for plugin authors
│   ├── schemas/
│   │   ├── plugin.manifest.schema.json §10.7  plugin manifest JSON schema
│   │   └── tup.manifest.schema.json    §10.7  Trust Update Protocol manifest schema
│   ├── seed_roots/root_ca.pem          §10.7  root CA for trust chain bootstrap
│   └── templates/                      §10.7  starter scaffolds for new plugins
│       ├── cpp/layer/                  C++ layer plugin template
│       ├── python/layer/               Python layer plugin template
│       └── generate_manifest.py        manifest generation helper
│
├── plugins/                            ← Reference / built-in plugin implementations
│   ├── bpe_tokenizer/                  §10.8  BpeTokenizerPlugin.cpp + manifest
│   └── example_activation/            §10.8  ExampleActivationPlugin.cpp + manifest
│
├── python-bridge/                      §10.9
│   ├── bindings/module.cpp             pybind11 C++ extension module entry point
│   ├── pyproject.toml
│   └── nnstudio/                       Python package (torch-style import)
│       ├── __init__.py
│       ├── torch_compat.py             Python-side torch shims
│       ├── plugin_manifest.py
│       ├── keras/                      §10.9  Keras façade (Sequential, compile, fit)
│       │   ├── __init__.py
│       │   ├── _model.py
│       │   ├── layers.py
│       │   ├── losses.py
│       │   ├── optimizers.py
│       │   └── callbacks.py
│       └── runners/                    §10.9  inference clients
│           ├── base.py                 RunnerBase contract
│           ├── onnx_runtime.py
│           ├── tf_serving.py
│           ├── triton.py
│           ├── kserve.py
│           └── openai.py
│
└── tests/
    ├── core/
    │   ├── test_tensor.cpp             Ch.1,8
    │   ├── test_trainer_xor.cpp        Ch.8   XOR milestone (annotated)
    │   ├── test_compute_graph.cpp      Ch.4
    │   ├── test_checkpoint.cpp         §10.5
```

```
│   ├── test_early_stopping.cpp     §10.5
│   └── test_compat.cpp             §10.6
├── builtin/
│   ├── test_activations.cpp        Ch.3 §10.1
│   ├── test_layers.cpp             Ch.3 §10.2
│   ├── test_conv2d.cpp             §10.2
│   ├── test_embedding.cpp          §10.2
│   ├── test_losses.cpp             Ch.5
│   └── test_optimizers.cpp         Ch.6
├── compat/
│   └── test_torch_compat.cpp       §10.6
└── plugin-api/
    ├── test_plugin_loader.cpp      §10.7
    ├── test_bpe_tokenizer.cpp      §10.8
    ├── test_trust_store.cpp        §10.7
    └── test_trust_verifier.cpp     §10.7
```

Appendix B — Namespace Map and Naming Conventions

> **TODO:** Expand this section into a standalone `NAMING-CONVENTIONS.md`.

The three-tier namespace design (target state)

Three visibility tiers separate the stable public contract from engine internals from bundled implementations.

Tier 1 — `nnstudio::core::` — the stable, versioned public contract

Contains **only** abstract extension points and core data types.
No concrete implementation ever lives here.
Plugin authors include `<core/ILayer.h>` etc. and work entirely inside `nnstudio::core::`.
Breaking changes require a version bump and a transition period.

```
nnstudio::core::
    ILayer          abstract layer contract (build/forward/backward/zeroGrad)
    IBackend        abstract compute contract (matmul, transpose, elementwise...)
    ILoss           abstract loss contract
    IOptimizer      abstract optimizer contract
    ILRScheduler    optional learning-rate schedule contract
    Tensor          core data type (not NN-specific; used by all tiers)
    Parameter       { name, Tensor, frozen } — the unit the optimizer reads
    Result<T>       error-or-value return type
    Shape           vector<int64_t> alias
    Device          CPU | CUDA | QUANTUM enum
    DType           float32 | float16 | int8 enum
    BackendRegistry singleton registry + backend() free function
    PluginDescriptor  C-ABI plugin descriptor (name, version, type tag, factory)
    FeatureFlags    FREE | PRO | ENTERPRISE tier gating
    Trainer         training loop (DataBatch, Dataset, TrainMetrics, TrainCallbacks)
    Sequential      ordered container of ILayer instances
```

Tier 2 — `nnstudio::internal::` — engine guts, not for plugin authors

Contains the engine's own machinery.
Documented as: *"do not use from plugins — may change in any release with no warning."*

```
nnstudio::internal::
    graph::         ComputeGraph, autograd traversal, DAG node types
    training::      Trainer step loop, callback dispatch, EvalTrace capture
    formats::       .nnsp / .nnsx file I/O, ONNX serialisation
    detail::        utility templates, helpers, type traits
```

Tier 2b — `nnstudio::ui::` — UI extension points (Phase 3+)

Semi-stable; evolves with Qt version requirements.
Plugin authors use this only when registering a custom UI panel.

```
nnstudio::ui::
    panels::        QML panel plugin registration interface
    qml::           QML-side property/signal contract types
```

Tier 3 — `nnstudio::builtin::` — NNStudio's own implementations

NNStudio's default implementations are treated identically to third-party plugins.
They happen to ship bundled with the installer, but they have no special namespace or access privilege.
A list of all loaded layer types will show `nnstudio::builtin::layers::Dense` alongside `myplugin::layers::MyLayer` — no VIP.

```
nnstudio::builtin::
    layers::        Dense, ReLU, LeakyReLU, Sigmoid, Tanh, Softmax, GELU,
                    Conv2D, BatchNorm, Dropout, Embedding, ...
    backends::      CpuBackend, CudaBackend, QuantumBackend
    losses::        MSE, CrossEntropy, BinaryCrossEntropy, HuberLoss
    optimizers::    SGD, Adam, AdamW, RMSProp
```

A third-party plugin developer writes:

```
eu::plachy::nnplugins::myplugin::layers::MyLayer : public nnstudio::core::ILayer { ... }
```

NNStudio itself writes:

```
nnstudio::builtin::layers::Dense : public nnstudio::core::ILayer { ... }
```

Both are just implementations. The only allowed difference is that `builtin` ships by default and appears first in UI lists.

Current state vs target state

~~The current code does not yet match the target. This is the known delta:~~

Migration complete. All refactors below are applied to the on-disk source; all 16 `ctest` tests pass; `cmake --build` clean; old files deleted.

Phase 1 — builtin namespaces

Symbol	Old namespace (deleted)	New namespace ✅
`Dense`	`nnstudio::layers::`	`nnstudio::builtin::layers::`
`ReLU`, `Sigmoid`, etc.	`nnstudio::activations::`	`nnstudio::builtin::layers::` (activations are layers)
`MSE`, `BCE`, etc.	`nnstudio::losses::`	`nnstudio::builtin::losses::`
`SGD`, `Adam`, `AdamW`	`nnstudio::optimizers::`	`nnstudio::builtin::optimizers::`
`CpuBackend`	`nnstudio::`	`nnstudio::builtin::backends::`
`ILoss`	bundled in `Losses.h`	extracted to `nnstudio::core::ILoss` in `ILoss.h` (Tier 1)

Symbol	Old namespace (deleted)	New namespace ✅
`IOptimizer` , `ILRScheduler`	bundled in `Optimizers.h`	extracted to `nnstudio::core::IOptimizer` in `IOptimizer.h` (Tier 1)

Phase 2 — core namespace + Option B folder structure

Symbol	Old namespace (deleted)	New namespace ✅
`Tensor` , `Parameter` , `Shape` , `Device` , `DType`	`nnstudio::`	`nnstudio::core::`
`ILayer` , `Sequential` , `LayerPtr`	`nnstudio::`	`nnstudio::core::`
`IBackend` , `BackendRegistry`	`nnstudio::`	`nnstudio::core::`
`ILoss` , `IOptimizer` , `ILRScheduler`	`nnstudio::`	`nnstudio::core::`
`Trainer` , `DataBatch` , `Dataset` , `TrainMetrics`	`nnstudio::`	`nnstudio::core::`
`Result<T>` , `FeatureFlags`	`nnstudio::`	`nnstudio::core::`
All files	`core/include/nnstudio/*.h` + `builtin/include/nnstudio/**/*.h`	`include/core/*.h` + `include/builtin/**/*.h` (Option B shared root)

Builtin `include/` and `src/` files gained `using namespace nnstudio::core;` inside (or just before) their namespace block so they reference Tier 1 types without qualification.

The path = namespace rule

Every on-disk path segment translates directly to exactly one C++ namespace segment — with two exceptions that are **visibility boundaries only**, never namespace contributors: `include/` and `src/` .

```
Disk path segment          →   C++ namespace segment
──────────────────────────────────────────────────────────
nnstudio/                  →   nnstudio::           (repo root = outermost namespace)
include/  or  src/         →   (SKIP — visibility boundary, not a namespace layer)
core/                      →   core::
builtin/                   →   builtin::
layers/                    →   layers::
backends/                  →   backends::
losses/                    →   losses::
optimizers/                →   optimizers::
Tensor.h                   →   class/namespace Tensor  (contents of the file)
──────────────────────────────────────────────────────────
Rule:  EVERY segment except include/ and src/ maps 1-to-1 to a namespace segment.
```

Examples:

```
nnstudio / include / core    / Tensor.h
    ↓       (skip)    ↓           ↓
nnstudio::           core::      Tensor

nnstudio / include / builtin / layers / Dense.h
    ↓       (skip)     ↓          ↓         ↓
nnstudio::            builtin:: layers::  Dense

nnstudio / src     / core    / Tensor.cpp
    ↓       (skip)    ↓           ↓
nnstudio::           core::      (Tensor impl — same namespace, visibility-only boundary)
```

This is the convention enforced by the Option B shared-root layout. See the **Plugin exception** section below for the one deliberate reversal of this rule.

Folder structure (current state)

The folder layout reflects namespace tiers, the one-directional mirror rule, and the colocated-headers-for-implementations principle.

```
nnstudio/
    include/                          ← ONE CMake search root (all targets: core, builtin, …)
        core/                         → namespace nnstudio::core::
            Tensor.h                  → nnstudio::core::Tensor
            Layer.h                   → nnstudio::core::ILayer, Sequential, LayerPtr, Parameter
            IBackend.h                → nnstudio::core::IBackend
            BackendRegistry.h         → nnstudio::core::BackendRegistry
            ILoss.h                   → nnstudio::core::ILoss
            IOptimizer.h              → nnstudio::core::IOptimizer, ILRScheduler
            Trainer.h                 → nnstudio::core::Trainer, DataBatch, Dataset, TrainMetrics
            Result.h                  → nnstudio::core::Result<T>
            Device.h  DType.h         → nnstudio::core::Device, DType
            FeatureFlags.h            → nnstudio::core::FeatureFlags
        builtin/                      → namespace nnstudio::builtin::
            layers/                   → nnstudio::builtin::layers::
                Dense.h
                Activations.h
            backends/                 → nnstudio::builtin::backends::
                CpuBackend.h
            losses/                   → nnstudio::builtin::losses::
                Losses.h
            optimizers/               → nnstudio::builtin::optimizers::
                Optimizers.h

    src/                              ← mirrors include/ exactly — build-only, never installed
        core/
            Tensor.cpp  BackendRegistry.cpp  Layer.cpp  Trainer.cpp
        builtin/
            layers/     Dense.cpp  Activations.cpp
            backends/   CpuBackend.cpp
            losses/     Losses.cpp
            optimizers/ Optimizers.cpp

    core/                             ← CMake target definition only
        CMakeLists.txt                ← target nnstudio-core; sources from ../src/core/
        CONTRIBUTING.md

    builtin/                          ← CMake target definition only
        CMakeLists.txt                ← targets nnstudio-builtin + nnstudio-backend-cpu
        CONTRIBUTING.md

    plugins/                          ← third-party and first-party plugin slots
        README.md
        CONTRIBUTING.md
        (see Plugin exception below for layout rules inside this folder)

    tests/
        core/
            test_tensor.cpp
            test_activations.cpp
            test_trainer_xor.cpp
```

The mirror rule (one-directional):

Every subfolder present in `include/` **must** have a corresponding subfolder in `src/`.
The reverse does not hold — `src/` may gain private subfolders (e.g. `src/graph/`, `src/training/`) that have no counterpart in `include/` when they are build-internal only.

Plugin authors add `${nnstudio_SOURCE_DIR}/include` to their include search path. Nothing else.

Per-folder documentation files:
Every folder that contains primarily subfolders (rather than files) carries two Markdown files:

- `README.md` — what this folder contains in terms of *software concept* (what it does at runtime)
- `CONTRIBUTING.md` — why this folder exists *here*, why it is structured this way, architectural decisions (the CUDA-linking argument, the curated-install argument, the intended reading order of subfolders as a numbered list under a `## Reading order` heading)

On `interfaces/` subfolders and why collocation with plural names is better

The C++ community argument against a separate `interfaces/` folder: it mechanically separates every `IFoo.h` from every `FooImpl.h` into parallel trees — a separation that adds navigation work but communicates nothing the `I` prefix doesn't already convey.

The `include/` tree above is different: it is the **publicly installed plugin SDK surface** — a boundary between what is stable and what is internal. The folder name is `include/` (universal tooling convention), not `api/` (which implies a REST or binding layer to most readers). Inside `include/`, plural subfolder names (`layers/`, `backends/`, …) mirror `src/` subfolders, making navigation predictable. This is what the Android NDK, LLVM, Qt, and virtually every major C++ SDK does: installed headers in a predictable include tree vs implementation files that never leave the build.

Naming rules summary

Element	Convention	Example
Interface / abstract base	`I` prefix	`ILayer`, `IBackend`
Concrete implementation	Descriptive name only	`Dense`, `Adam`, `CpuBackend`
Path → namespace	Every path segment (except `include/` / `src/`) maps 1-to-1 to a namespace segment	`include/core/Tensor.h` → `nnstudio::core::Tensor`
Namespace for plugin implementations	Author's reverse-domain identity (**not** derived from `plugins/` folder position)	`eu::plachy::nnplugins::myplugin::layers::MyLayer`
Plugin distribution ID (manifest)	reverse-domain dot-notation	`"id": "eu.plachy.nnplugins.myplugin"`
Filename	PascalCase, no prefix/suffix	`Dense.h`, `CpuBackend.cpp`
`_` prefix in filenames	**Do not use** — reserved/confusing in C++	—
Folder names (domain collections)	Plural by convention	`layers/`, `backends/`, `losses/`
Folder names (single-concept roots)	Singular or plural as natural language dictates	`plugins/`, `formats/`
`include/` subfolder → `src/` subfolder	One-directional mirror: every `include/X/` implies `src/X/` exists	`include/core/` ↔ `src/core/`
`src/` private subfolder	May exist with no `include/` counterpart	`src/graph/`, `src/training/`
`using namespace nnstudio::core;` in builtin	Placed after innermost namespace opening in `.h`; at file scope before namespace block in `.cpp`	`namespace nnstudio::builtin::layers { using namespace nnstudio::core; … }`
Per-folder documentation	`README.md` (software concept) + `CONTRIBUTING.md` (repo structure rationale + `## Reading order` list)	—

The `I` prefix on the filename is the complete interface signal — no separate `interfaces/` subfolder is needed. The `include/` vs `src/` split already expresses the contract-vs-implementation distinction at the directory level.

Plugin exception — the rule is deliberately reversed for `plugins/`

All rules above derive one direction: **folder path → C++ namespace**. Inside `plugins/` this direction is **inverted**: the author's namespace identity is the primary artifact, and the on-disk folder layout follows from it.

Why?
The namespace of a plugin is owned by its author, not by NNStudio. It is based on the author's reverse-domain identity and must not change when the plugin moves between repositories, organizations, or deployment targets. Deriving it from the folder position inside `nnstudio/plugins/` would couple it to NNStudio's internal directory structure — the exact coupling the plugin system exists to avoid.

Layout inside `plugins/` :

```
nnstudio/plugins/
    <plugin-slug>/                  ← reverse-domain slug, e.g. eu.plachy.nnplugins.myplugin
        include/                    ← visibility boundary (same semantics as everywhere else)
            layers/                 → author-controlled namespace segment
                MyLayer.h
            backends/
                MyBackend.h
            ui/
                MyPanel.h
        src/                        ← build-only mirror of include/
            layers/
                MyLayer.cpp
            backends/
                MyBackend.cpp
            ui/
                MyPanel.cpp
        CMakeLists.txt
        README.md
        CONTRIBUTING.md
```

Namespace ownership:
The `<plugin-slug>` folder name does **not** contribute a namespace segment. The plugin author decides the full namespace independently:

```
plugins / eu.plachy.nnplugins.myplugin / include / layers / MyLayer.h
 (skip)          (slug — skip)           (skip)      ↓          ↓
                                                  layers::      MyLayer
```

The segments above `layers/` are entirely the author's choice, e.g.:

```
// Author chooses their own top-level namespace:
namespace eu::plachy::nnplugins::myplugin::layers {
    class MyLayer : public nnstudio::core::ILayer { … };
}
```

Matching table — core/builtin vs plugins:

Aspect	`core/` and `builtin/`	`plugins/<slug>/`
Namespace derived from	folder path (forward rule)	author identity (reverse rule)
Folder slug contributes namespace segment?	Yes — `core/` → `::core::` , `builtin/` → `::builtin::`	No — slug is an artifact identifier, not a namespace
`include/` / `src/` are visibility boundaries?	Yes	Yes (same)

Aspect	`core/` and `builtin/`	`plugins/<slug>/`
Domain subfolders (`layers/` , `backends/` , …) contribute namespace?	Yes	Yes — same sub-segment rule applies *within* the plugin
Inherits from	`nnstudio::core::ILayer` , etc.	`nnstudio::core::ILayer` , etc. (same)

> **Current status (documentation only):** The `plugins/` folder exists as a placeholder. Moving `builtin/` content into `plugins/builtin/` is a future option that requires no namespace changes — only file relocation and CMakeLists updates.

Appendix C — Architecture Templates

> 🥪 These are the "sandwich presets" in the UI. Each is a proven starting point, not a law.
> The engine imposes no template; the only hard rule is: **no two consecutive Dense layers without a non-linear layer between them**.
> See `blueprints.md §3.9` for the mathematical proof of why.

Template 1 — MLP (Multi-Layer Perceptron)

Use for: tabular data, binary/multi-class classification, regression.

```
Input → Dense(n_hidden) → ReLU → [Dense(n_hidden) → ReLU]... → Dense(n_out) → output activation
```

Output activation by task:

- Binary classification → Sigmoid + BCE loss
- Multi-class → Softmax + CrossEntropy loss
- Regression → none (linear output) + MSE loss

Typical sizes:

Network	Layer widths	Params
XOR (our test)	2→4→1	13
Small classifier (MNIST)	784→512→256→10	~670 K
Medium tabular	64→256→256→128→1	~130 K

Status in NNStudio: fully operational. XOR is the proof.

Template 2 — Autoencoder

Use for: compression, anomaly detection, denoising, unsupervised feature learning.

```
Input → Dense(512)→ReLU → Dense(256)→ReLU → Dense(bottleneck)→ReLU   ← encoder
      → Dense(256)→ReLU → Dense(512)→ReLU → Dense(input_size)→Sigmoid  ← decoder
```

The loss is reconstruction: MSE or BCE between output and original input.
`target = input` — self-supervised, no external labels needed.

Status in NNStudio: engine supports it (all required layers and losses exist); ILoss contract uses an external `target` argument so you pass the input batch as both `inputs` and `targets` in `trainStep()` .

Template 3 — CNN (Convolutional Neural Network)

Use for: images, audio spectrograms, any data with spatial locality.

```
Input → [Conv2D → ReLU → MaxPool]... → Flatten → Dense(256) → ReLU → Dense(n_classes) → Softmax
```

Conv2D layers detect local patterns (edges, textures) independent of position.
MaxPool reduces spatial size. Flatten converts 2D feature maps to 1D for the Dense head.

Typical sizes:

Network	Architecture	Params
LeNet-5 (1998)	2×Conv + 2×Dense	~60 K
AlexNet (2012)	5×Conv + 3×Dense	~60 M
ResNet-50 (2015)	50 layers, skip connections	~25 M

Status in NNStudio: `Conv2D` not yet implemented. `ILayer` interface supports it; Phase 1 TODO.

Template 4 — Transformer block (encoder or decoder)

Use for: text, language understanding, generation, reasoning, long-range dependencies.

One Transformer block (stacked N times):

```
x → LayerNorm → MultiHeadSelfAttention → + x   (residual / skip connection)
  → LayerNorm → Dense(4×d_model)→GELU→Dense(d_model) → + x   (FFN block + residual)
```

The full GPT-style model:

```
Token embedding + positional encoding
→ N × Transformer block
→ LayerNorm
→ Dense(d_model → vocab_size)   (the "language model head")
```

Layer-by-layer breakdown

> All four layer types below are **not yet in NNStudio**. They are explained here so the design intent is clear when they are implemented.

4a. Embedding

What it is: a learnable lookup table.

Words (tokens) arrive as integers — indices into a vocabulary of size `V` (e.g. 50 257 for GPT-2). The network cannot process raw integers directly; it needs real-valued vectors it can do linear algebra on. An embedding layer is simply a weight matrix `E` of shape `[V, d_model]`. A forward pass for token index `i` returns row `i` of `E` — that is it.

```
token id = 42  →  E[42]  →  vector of shape [d_model]
```

No sigmoid, no dot product. Just a lookup. But because `E` is a weight matrix, backprop tunes it: after training, tokens with similar meaning end up with nearby vectors (this is where "word2vec-style geometry" comes from: `king − man + woman ≈ queen`).

Parameters: `V × d_model` weights. For GPT-2 small: `50 257 × 768 ≈ 38.6 M` parameters — the embedding table alone is 33% of the model.

Status in NNStudio: not implemented. `ILayer::build(Shape)` would need to accept a scalar integer index as input shape, which breaks the current Tensor contract (which expects float tensors). This needs a design decision before implementing.

4b. Positional encoding

Why it is needed: attention (see §4c) treats the input as a *set* — it has no idea which token came first. Position must be injected explicitly.

Two approaches exist:

Learned (GPT-2, BERT): another embedding table `PE` of shape `[max_sequence_length, d_model]`. Position `p` maps to `PE[p]`. Added to the token embedding: `x = E[token] + PE[position]`.

Sinusoidal (original 2017 Transformer): a fixed formula, no learned parameters:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Different frequency sine/cosine waves fill alternating dimensions. The result: each position has a unique vector, and nearby positions have similar vectors. Modern models (Llama) use **RoPE** (Rotary Position Embedding) instead — applied inside the attention computation rather than at input.

Status in NNStudio: not implemented. Sinusoidal PE is parameter-free (just an `apply()` function on the input tensor) and could be implemented before the embedding layer.

4c. MultiHeadSelfAttention

This is the core invention of the Transformer. It replaces recurrence entirely.

The problem attention solves: in a sentence like "The animal didn't cross the street because *it* was too tired", deciding what "it" refers to requires comparing every token to every other token simultaneously. An RNN has to thread that information through a bottleneck hidden state. Attention makes all token-to-token comparisons in one step.

Single-head attention:

Each token vector `x_i` is projected into three roles through three learned weight matrices `W_Q`, `W_K`, `W_V` (all `[d_model, d_k]`):

- **Query** Q: "what am I looking for?" — `Q = x W_Q`
- **Key** K: "what do I contain?" — `K = x W_K`
- **Value** V: "what do I contribute?" — `V = x W_V`

The attention score between position `i` (query) and position `j` (key) is their dot product, scaled:

$$A_{ij} = \frac{Q_i \cdot K_j}{\sqrt{d_k}}$$

The $\sqrt{d_k}$ scaling prevents the dot products from growing large and pushing softmax into its flat saturation region (vanishing gradients). Softmax across all `j` makes the scores sum to 1 — they become mixing weights. The output for token `i` is a weighted sum of all value vectors:

$$\text{out}_i = \sum_j \text{softmax}(A_{ij}) \cdot V_j$$

In matrix form (all tokens at once):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Multi-head: instead of one set of (Q, K, V) matrices, run `n_heads` of them in parallel, each with `d_k = d_model / n_heads`. Each head can attend to different aspects (syntax, coreference, topic). Outputs are concatenated and projected back:

$$\text{MultiHead}(x) = \text{concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O$$

Parameters per attention block: `4 × d_model²` — the four matrices W_Q, W_K, W_V, W_O. For `d_model=768`: `4 × 768² = 2.36 M` per block.

Causal mask (decoder only): in a GPT-style model, position `i` must not attend to position `j > i` (future tokens). This is enforced by adding `-∞` to the attention scores before softmax for all future positions — they become 0 after softmax. This is a single mask operation, not a structural change.

Status in NNStudio: not implemented. Requires:

- The QKV projection (three Dense layers per head, or one fused Dense of `3×d_model`)
- Scaled dot product (Tensor batch-matmul operation, not yet in the Tensor API)
- Softmax over a variable-length dimension
- Output projection Dense
- Optional causal mask (Phase 3)

4d. LayerNorm

What it is: normalization applied independently to each token's vector.

After attention and FFN operations, activation magnitudes can drift — some vectors become very large, some very small. This destabilizes subsequent layers. Normalization is the fix.

Formula: for a single vector `x` of length `d_model`:

$$\hat{x}_i = \frac{x_i - \mu}{\sigma + \varepsilon}, \quad \mu = \frac{1}{d}\sum_i x_i, \quad \sigma = \sqrt{\frac{1}{d}\sum_i (x_i - \mu)^2}$$

Then apply learned scale `γ` and shift `β` (both vectors of length `d_model`):

$$y_i = \gamma_i \hat{x}_i + \beta_i$$

`γ` and `β` are initialized to 1 and 0, meaning "do nothing initially". They are learned weights — the optimizer adjusts them so the normalization doesn't remove useful structure.

Why not BatchNorm? BatchNorm normalizes across the batch dimension (per-feature statistics). For sequences this is unstable: batch size varies, sequences have different lengths, and during inference you might process one token at a time. LayerNorm normalizes within a single vector — no dependency on the rest of the batch.

Parameters: `2 × d_model` (one `γ` and one `β` vector). Tiny — for `d_model=768` that is 1 536 floats.

Status in NNStudio: not implemented. Forward pass is a pure tensor operation (mean, std, elementwise scale+shift). Backward requires the gradient through mean and variance — slightly involved but closed-form. A good early implementation target because it has no exotic topology requirements.

4e. GELU (activation)

What it is: Gaussian Error Linear Unit — the standard activation in the FFN block of modern Transformers.

$$\mathrm{GELU}(x) = x \cdot \Phi(x)$$

where $\Phi(x)$ is the standard Gaussian cumulative distribution function. Because computing Φ exactly requires numerical integration, a fast approximation is used in practice:

$$\mathrm{GELU}(x) \approx 0.5 \cdot x \cdot \left(1 + \tanh\left(\sqrt{\tfrac{2}{\pi}}\,(x + 0.044715\,x^3)\right)\right)$$

Compared to ReLU:

- ReLU: hard zero for `x < 0`, linear for `x > 0`. The sharp kink at 0 can cause neurons to "die" (stuck at zero gradient permanently).
- GELU: smoothly tapers near zero instead of clipping. Slightly negative values still pass a fractional signal through. Gradient exists everywhere.

Practically the difference is small in shallow networks. In very deep Transformers with 96+ layers (GPT-3) the accumulated smoothness advantage compounds.

Status in NNStudio: GELU can be implemented as an `IActivation` right now — it is a pure elementwise operation, same contract as ReLU/Sigmoid. The only thing needed is the math in `apply()` and the derivative `Φ(x) + x·ϕ(x)` (where `ϕ` is the Gaussian PDF) in `gradient()`.

Full data flow, annotated

```
Input: sequence of T token IDs  [T]  (integers)

[Embedding]      x = E[token_ids]                    shape: [T, d_model]
[+PE]            x = x + PE[0:T]                     shape: [T, d_model]

For each of N blocks:
  [LayerNorm]    x_norm = LN(x)                      shape: [T, d_model]
  [Attention]    attn = MultiHead(Q=x_norm, K=x_norm, V=x_norm)  shape: [T, d_model]
  [+ residual]   x = x + attn                        shape: [T, d_model]

  [LayerNorm]    x_norm = LN(x)                      shape: [T, d_model]
  [Dense→GELU]   h = GELU(x_norm @ W1 + b1)          shape: [T, 4×d_model]
  [Dense]        ffn = h @ W2 + b2                   shape: [T, d_model]
  [+ residual]   x = x + ffn                         shape: [T, d_model]

[LayerNorm]      x = LN(x)                           shape: [T, d_model]
[Dense head]     logits = x @ W_vocab + b_vocab       shape: [T, vocab_size]
[Softmax]        probs = softmax(logits[-1])          shape: [vocab_size]
                 → sample or argmax next token
```

Key architectural choices:

- `d_model` = embedding dimension (width of every token vector throughout the network)
- `n_heads` = number of parallel attention heads (each sees a `d_model/n_heads` subspace)
- FFN hidden is always `4×d_model` wide (convention from the 2017 paper; still used unchanged)
- GELU standard activation in FFN; ReLU also works
- Every sub-block has a residual/skip connection — this is why stacking 96 blocks is stable

Typical sizes for text reasoning / QA:

Model	d_model	Heads	Layers	Params	Context window
GPT-2 small	768	12	12	117 M	1 024 tokens
GPT-2 medium	1 024	16	24	345 M	1 024 tokens
GPT-2 XL	1 600	25	48	1.5 B	1 024 tokens
GPT-3	12 288	96	96	175 B	2 048 tokens
Llama 2 7B	4 096	32	32	7 B	4 096 tokens
Llama 2 70B	8 192	64	80	70 B	4 096 tokens
GPT-4 (est.)	~16 384 (MoE)	~128	~120	~1.8 T (MoE)	8 192+ tokens

For NNStudio's text parsing / reasoning / QA use case:
A Llama 2 7B-class model is the practical minimum for reliable open-domain reasoning. That is 7 billion floats × 4 bytes = **28 GB** just for weights in fp32; 14 GB in fp16. Training from scratch is not realistic — fine-tuning a pre-trained checkpoint (LoRA or full fine-tune) is the standard approach. NNStudio would load an existing `.onnx` or `.nns` checkpoint and run inference or fine-tuning on specific layers (`frozen = true` on base weights, `frozen = false` on LoRA adapters).

Status in NNStudio (updated — Phase 2 complete):

Layer	Status	Notes
`Embedding`	✅ implemented	Float-stored IDs; dedicated int DType is Phase 4
`LayerNorm`	✅ implemented	Normalises over last dim; learnable γ, β
`GELU`	✅ implemented	`GELUFn` (`IActivation`) + `GELU : ActivationBase`

Layer	Status	Notes
`MultiHeadAttention`	✅ implemented	Causal mask, 4 weight + bias pairs, full backward
`ComputeGraph` (skip connections)	❌ Phase 3	`Sequential` is still strictly linear
FFN (Dense→GELU→Dense)	✅	Both `Dense` and `GELU` ready; wire in `Sequential`

Overall: **all individual building blocks ready.** Residual connections (skip connections) require `ComputeGraph` (Phase 3). See §10.10 for the full updated status table.

Template 5 — ResNet block (skip connection)

Use for: deep networks where vanilla MLP would suffer vanishing gradients.

```
x → Dense(n) → ReLU → Dense(n) → + x  ← skip: add original x back
                                    → ReLU
```

The skip connection (residual connection) routes the gradient directly around the block. Even if the block adds near-zero contribution, the gradient highway through the skip ensures early layers still receive a clean signal. This is why networks with hundreds of layers became trainable.

Status in NNStudio: requires `ComputeGraph` for the branching topology. `Sequential` cannot express a skip connection (it is strictly linear). Phase 3.

On extending `ILayer` for genuinely new layer types

Plugging in a new activation: straightforward (Phase 2 `IActivation`, ADR-020).

Plugging in a new *structural* layer type (Conv2D, MultiHeadAttention, custom graph node): the `ILayer` interface itself is the extension point, but the downstream impact is non-trivial:

Component affected	Impact
`ILayer::forward/backward`	Must be implemented correctly including `lastInput_` / context management
`ILayer::build(Shape)`	Must return correct output shape so the shape-relay chain works
`ILayer::parameters()`	Must expose all learnable parameters for the Optimizer
`ComputeGraph`	New layer must register its ops as graph nodes (Phase 3)
`ONNX export`	Must map to an ONNX op-node or a custom op (Phase 4)
Studio UI	Need a property form, a topology diagram node, KB help entry
Plugin SDK	Must be distributable as a signed `.nnsp` plugin (Phase 2 ABI freeze)

This is not blocked — it is the intentional design. But it means adding a genuinely new structural layer type is a multi-phase commitment, not a weekend task. The engine is correctly designed for it; the scope is simply larger than adding an activation.

Appendix D — IBackend Vtable Reference

> **Why this is an appendix and not in Chapter 2:**
> Chapter 2 covers the *why* of `IBackend` — the Strategy pattern, the row/col-major trick, the single-call backend swap. This appendix covers the *what*: every virtual method with its full mathematical specification, a worked micro-example, and the practical question of which parts of the engine benefit from a CUDA backend. Most readers can skip this on first reading; it is the reference for backend implementors and for understanding ADR-034 step 2.

How this list was obtained

Every entry corresponds directly to a `virtual` method in `include/core/IBackend.h` — the canonical ABI contract. The file was read in full; nothing is inferred from documentation elsewhere. When `IBackend.h` changes (new method added, signature changed), this appendix must be updated to match.

§D.1 — Identity and Memory Group

These four methods are foundational plumbing that every backend — CPU, CUDA, remote, or quantum — must implement.

Method	Signature (simplified)	Purpose
`name`	`() → string_view`	Returns the registration key, e.g. `"cpu"`, `"cuda"`. This string is the argument to `BackendRegistry::setActive(name)`.
`device`	`() → Device`	Returns `Device::CPU`, `Device::CUDA`, or `Device::QUANTUM`. Tensors compare their device tag to avoid redundant copies in `to()`.
`alloc`	`(count: int64) → shared_ptr<float[]>`	Allocates a flat float buffer of `count` elements on the backend's native device. CPU: `new float[count]`. CUDA: `cudaMallocManaged`. Remote: allocates a handle on the remote node whose lifetime is tied to the returned shared_ptr.
`to`	`(tensor) → Tensor`	Copies or moves a tensor to this backend's device. CPU→CPU: may return the same buffer via reference counting (effectively free). CPU→CUDA: schedules a `cudaMemcpy H→D`. CUDA→CPU: `cudaMemcpy D→H` + `sync`.

Micro-example:

```
// CpuBackend active.  All on CPU, so `to` is a no-op:
auto buf = B().alloc(6);           // float[6] on the heap
Tensor a({2, 3}, buf);             // 2×3 matrix, shares buf
Tensor b = B().to(a);              // no-op: already CPU — same data pointer
assert(b.data() == a.data());      // ← true on CpuBackend
```

§D.2 — BLAS Level: `matmul`

This is the single most important vtable function. The entire performance of `Dense`, `MultiHeadAttention`, and (eventually) `Conv2D` flows through this one call.

Signature: `matmul(a: Tensor, b: Tensor) → Tensor`

Mathematical specification — 2-D case:

$$C_{[M,N]} = A_{[M,K]} \cdot B_{[K,N]}, \quad C_{ij} = \sum_{k=0}^{K-1} A_{ik} \cdot B_{kj}$$

The result shape is always `[M, N]`. Dimension K is the contracted (inner) dimension and must match between `a` and `b`.

3-D batched case:

$$C_{[B,M,N]} = A_{[B,M,K]} \cdot B_{[B,K,N]}, \quad C_{b,i,j} = \sum_{k=0}^{K-1} A_{b,i,k} \cdot B_{b,k,j}$$

Each slice along the batch dimension is an independent GEMM.

Numeric micro-example:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

$$A \cdot B = \begin{pmatrix} 1{\cdot}5 + 2{\cdot}7 & 1{\cdot}6 + 2{\cdot}8 \\ 3{\cdot}5 + 4{\cdot}7 & 3{\cdot}6 + 4{\cdot}8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

How `Dense` uses it:

```
forward:   y [B, outF] = B().matmul( x [B, inF],  Wt [inF, outF] )   // Wt = W^T
backward:  dW[outF,inF] = B().matmul( gT[outF,B],  x  [B,  inF]  )   // gT = gradOut^T
           dX[B,  inF ] = B().matmul( g [B, outF], W  [outF,inF]  )
```

CpuBackend implementation note: Uses the row/col-major transposition trick from Chapter 2. The `Eigen::Map` views are constructed in reversed argument order so Eigen's column-major SGEMM produces the correct row-major result without any data copy or transposition of the underlying buffer.

§D.3 — Element-wise Arithmetic Group

All methods here operate **element-by-element** on identically shaped tensors, or on one tensor and a scalar constant. They are entirely independent on each element — perfectly parallelisable (one GPU thread per element).

> **Critical distinction — Hadamard product vs matrix multiply:**
>
> `mul(a, b)` is the **Hadamard product** (denoted $A \odot B$) — each element at position i of `a` is multiplied by the element at the *same position* i of `b`. **Both operands must have exactly the same shape.** This is entirely different from `matmul`, which contracts an inner dimension and requires `A: [M,K]` and `B: [K,N]` (the rule "columns of A = rows of B"):
>
> | Operation | Shape rule | Result shape |
> |---|---|---|
> | `matmul(A, B)` | A is `[M,K]`, B is `[K,N]` — inner (K) dims must match | `[M,N]` = `[2,2]` |
> | `mul(A, B)` Hadamard | A and B must be **the same shape** | `[2,2]` (same) |
>
> **matmul — weighted sums across K connections:**
>
> $$\begin{pmatrix}1 & 2\\3 & 4\end{pmatrix} \cdot \begin{pmatrix}5 & 6\\7 & 8\end{pmatrix} = \begin{pmatrix}1{\cdot}5+2{\cdot}7 & 1{\cdot}6+2{\cdot}8\\3{\cdot}5+4{\cdot}7 & 3{\cdot}6+4{\cdot}8\end{pmatrix} = \begin{pmatrix}19 & 22\\43 & 50\end{pmatrix}$$
>
> **Hadamard ⊙ — per-position scaling, no summation:**
>
> $$\begin{pmatrix}1 & 2\\3 & 4\end{pmatrix} \odot \begin{pmatrix}5 & 6\\7 & 8\end{pmatrix} = \begin{pmatrix}1{\cdot}5 & 2{\cdot}6\\3{\cdot}7 & 4{\cdot}8\end{pmatrix} = \begin{pmatrix}5 & 12\\21 & 32\end{pmatrix}$$
>
> `matmul` says "take a weighted sum across all K connections" — the neuron's summing rule.
> `mul` says "scale each position independently" — the dropout mask, the activation gradient gate.
>
> **Why does Hadamard require identical shapes?**
> Because $c_{ij} = a_{ij} \cdot b_{ij}$ — there is no summation; every output element needs exactly one pair. If shapes differ there is no 1-to-1 correspondence. Broadcasting (allowing mismatched shapes provided one can be stretched) is a separate capability not currently in `IBackend`; it would require a `mulBroadcast` method.
>
> **Note on the scalar variants** (`addScalar`, `subScalar`, `mulScalar`, `divScalar`): these belong in the same Arithmetic group rather than the Math Functions group (§D.4) because they form natural paired families with their tensor-tensor counterparts (`add` / `addScalar`, `mul` / `mulScalar`, etc.). Operationally they are equally "element-wise" — the grouping is structural convention for readability, not a mathematical distinction. `mulScalar(a, 2.0)` and `sqrt(a)` are equally per-element; they just appear in different rows because of their pairing origin.

Method	Math	Concrete example (flat tensors for brevity)	Used by
`add(a, b)`	$c_i = a_i + b_i$	`{1,2,3} + {4,5,6}` → `{5,7,9}`	Bias column-broadcast in `Dense`; residual skip connections (Phase 3)
`sub(a, b)`	$c_i = a_i - b_i$	`{5,7,9} - {3,2,1}` → `{2,5,8}`	Loss gradients: `pred - target`; BatchNorm centering
`mul(a, b)`	$c_i = a_i \cdot b_i$ — **Hadamard** (element i × element i, same-shape operands required)	`{2,4,6}` ⊙ `{1,0,1}` → `{2,0,6}`	Dropout mask application; ReLU/LeakyReLU backward gradient masking; attention weight × value
`div_(a, b)`	$c_i = a_i / b_i$ (name avoids C++ `div` keyword)	`{6,8,4}` / `{2,4,2}` → `{3,2,2}`	Sigmoid forward: `ones / (ones + exp(-x))`; LayerNorm normalisation

Method	Math	Concrete example (flat tensors for brevity)	Used by
`addScalar(a, s)`	$c_i = a_i + s$	`{1,2,3}` + 1.0 → `{2,3,4}`	Sigmoid: `exp(-x) + 1.0`; Adam bias-correction denominators `1 - β^t`
`subScalar(a, s)`	$c_i = a_i - s$	`{5,6,7}` − 2.0 → `{3,4,5}`	BatchNorm/LayerNorm: subtract mean
`mulScalar(a, s)`	$c_i = a_i \cdot s$	`{2,4,8}` × 0.5 → `{1,2,4}`	Sigmoid derivative: `mulScalar(sq, -1)` for `1 - tanh²`; dropout inverted scale `1/(1-p)`
`divScalar(a, s)`	$c_i = a_i / s$	`{6,8,4}` / 2.0 → `{3,4,2}`	Attention scale $1/\sqrt{d_k}$; MSE: divide sum by N
`neg(a)`	$c_i = -a_i$	`{1,-2,3}` → `{-1,2,-3}`	Sigmoid: `neg(x)` to get $-x$ before `exp`; BCE gradient sign flip

Anatomy of Sigmoid via vtable calls:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

```
// SigmoidFn::forward — entirely composed of IBackend methods:
Tensor neg_x   = B().neg(x);                    // step 1: -x
Tensor exp_neg = B().exp(neg_x);                 // step 2: e^{-x}
Tensor denom   = B().addScalar(exp_neg, 1.0f);   // step 3: 1 + e^{-x}
Tensor ones    = Tensor::ones(x.shape());
Tensor out     = B().div_(ones, denom);           // step 4: 1 / (…)
```

Every step dispatches through `IBackend`. A CUDA backend would execute all four operations as GPU kernels — no CPU involvement.

§D.4 — Element-wise Math Functions Group

Unary functions applied independently to each element. Like §D.3, they are trivially parallelisable.

Method	Math	Example	Notes
`exp(a)`	$c_i = e^{a_i}$	`exp({0, 1, 2})` → `{1.0, 2.718, 7.389}`	Sigmoid, Softmax, GELU. Overflow for $a_i \gg 80$; Softmax always applies max-subtraction first: $e^{x_i - \max_j x_j}$
`log(a)`	$c_i = \ln(a_i)$	`log({1.0, e, e²})` → `{0, 1, 2}`	CrossEntropy, BCE. $\ln(0) = -\infty$ **silently produces NaN.** All loss code clamps inputs: `clamp(p, 1e-7, 1-1e-7)` before calling `log`
`sqrt(a)`	$c_i = \sqrt{a_i}$	`sqrt({4.0, 9.0, 16.0})` → `{2, 3, 4}`	Adam update denominator $\sqrt{\hat{v}_t} + \varepsilon$; prevents division by zero when gradients are very small
`abs(a)`	$c_i = \lvert a_i \rvert$	`abs({-3, 4, -0.5})` → `{3, 4, 0.5}`	HuberLoss: compares $\lvert p - t \rvert$ against the threshold δ to choose the quadratic vs. linear branch
`clamp(a, lo, hi)`	$c_i = \min(\max(a_i, \text{lo}), \text{hi})$	`clamp({-2, 0.5, 3}, 0, 1)` → `{0, 0.5, 1}`	ReLU forward: `clamp(x, 0, +∞)` (most elegant use — entire ReLU is one vtable call); `kEps` loss guard: `clamp(p, 1e-7, 1-1e-7)`

> **`clamp` by other names — same concept appears across many fields:**
>
> - **Audio engineering / sound mixing:** *hard clipping* or *peak limiting* — any signal level exceeding the ceiling is cut flat; the louder the input beyond the threshold, the more is lost (unlike soft-knee compression which is gentle). `clamp(signal, -1.0, 1.0)` is exactly what a digital limiter does.
> - **Electronics / signal processing:** *saturation* — an amplifier that cannot output beyond its supply voltage clips; *hard limiting*; or *rail-to-rail clipping* in ADC overflow.
> - **Image processing:** thresholding or *value quantisation floor/ceiling* — clamping pixel values to `[0, 255]` after a brightness adjustment.
> - **Control systems:** *output saturation* — a PID controller whose actuator has physical limits (a motor can't spin faster than its max RPM).

> In all cases the concept is identical: values inside the band `[lo, hi]` pass through unchanged; values outside are pinned to the nearest boundary. The NNStudio use is precisely the audio-engineer's "limiter for numerical safety": prevent `log(0)` → `-∞` by pinning predictions to `[1e-7, 1-1e-7]` before computing entropy.

`tanh` could be expressed as $\tanh(x) = 2\sigma(2x) - 1 = 2/(1+e^{-2x}) - 1$ — entirely via vtable ops. Instead it currently uses a raw C loop:

```
// TanhActFn::forward — NOT through IBackend:
for (int64_t i = 0; i < x.numel(); ++i)
    out.flat(i) = std::tanh(x.flat(i));   // CPU-only std::tanh
```

A CUDA backend would not accelerate this. The vtable rewrite is a one-liner once `IBackend` has `exp` (which it already does). This is an identified Phase 4 improvement.

§D.5 — Reduction Group

Reductions collapse one dimension (or all dimensions for `dim = -1`) to a single value.

Working matrix used in all examples below:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} \quad \text{shape } [3, 2]$$

The two axes are: `dim=0` = the row axis (collapse 3 rows → 1), `dim=1` = the column axis (collapse 2 columns → 1).

Method	Math	Examples with A `[3,2]` above	Notes
`sum(a, dim, keepdim)`	$c_j = \sum_i a_{ij}$ along `dim`	`sum(A, dim=0, false)` → `{9, 12}` shape `[2]` (col sums: 1+3+5, 2+4+6) · `sum(A, dim=1, false)` → `{3, 7, 11}` shape `[3]` (row sums: 1+2, 3+4, 5+6) · `sum(A, dim=-1, false)` → `{21}` shape `[1]` (global)	Bias gradient in `Dense::backward`: `sum(gradOut, dim=0)` accumulates all batch rows into one gradient vector
`mean(a, dim, keepdim)`	$c_j = \frac{1}{N}\sum_i a_{ij}$	`mean(A, dim=0, true)` → `{{3.0, 4.0}}` shape `[1,2]` (col means: (1+3+5)/3, (2+4+6)/3) · `mean(A, dim=1, false)` → `{1.5, 3.5, 5.5}` shape `[3]` (row means)	MSE loss output (global mean). `keepdim=true` keeps the collapsed dim as size 1 — needed for broadcasting subtraction: `A - mean(A, dim=0, keepdim=True)` zero-centres each column
`max(a, dim, keepdim)`	$c_j = \max_i a_{ij}$	`max(A, dim=0, false)` → `{5, 6}` shape `[2]` (col maxima) · `max(A, dim=1, true)` → `{{2},{4},{6}}` shape `[3,1]` (per-row max)	Numerically stable Softmax: subtract per-row max before exp — `max(A, dim=1, keepdim=True)` gives shape `[3,1]` which broadcasts cleanly against `A [3,2]`

Why `keepdim` matters — illustrated:

```
A:  shape [3, 2]
    | 1  2 |
    | 3  4 |
    | 5  6 |

mean(A, dim=0, keepdim=False) →  | 3.0  4.0 |   shape [2]   ← 1-D, no outer dimension kept
                                       ↕
                                 can't broadcast against A [3,2]: rank mismatch

mean(A, dim=0, keepdim=True)  →  | 3.0  4.0 |   shape [1, 2]   ← dim 0 kept as size-1
                                       ↕
                                 broadcasts against A [3, 2]: each row gets centred
                                       ↓
        A - mean(keepdim=True):  | -2.0  -2.0 |
                                 |  0.0   0.0 |
                                 |  2.0   2.0 |
```

Softmax numerical stability via `max` + `sum`:

$$\text{softmax}_i = \frac{e^{a_i - M}}{\sum_k e^{a_k - M}}, \quad M = \max_j a_j$$

Row example with `a = {1, 3, 2}`:

Without stability	With `M = max = 3`
$e^1 = 2.72,\ e^3 = 20.09,\ e^2 = 7.39 \rightarrow$ sum=30.2	$e^{-2} = 0.135,\ e^0 = 1.0,\ e^{-1} = 0.368 \rightarrow$ sum=1.503
softmax = `{0.090, 0.665, 0.245}` ✓	softmax = `{0.090, 0.665, 0.245}` ✓ (identical)
overflow safe? For large inputs: e^{800} = inf ✗	$e^{800-800} = e^0 = 1.0$ ✓ always safe

Subtracting M shifts all exponents so the largest is always $e^0 = 1.0$ — no overflow regardless of input magnitude. The result is mathematically identical because the e^{-M} factors cancel in numerator and denominator.

§D.6 — Shape Operations Group

These do not move elements in memory on CPU; they manipulate the `strides_` and `shape_` metadata only. On CUDA, a backend may need to produce contiguous copies if post-op kernels require NCHW-contiguous layout.

Method	What it does	Example	Notes
`reshape(a, newShape)`	Reinterpret memory as a different shape, preserving total element count (`numel`)	`reshape([6], {2,3})` — a flat vector becomes a 2×3 matrix	Strides-only alias if data is contiguous. Used by `Embedding` to pack batch output; MHA head split/merge
`transpose(a, dim0, dim1)`	Swap two dimensions — strides swap only, no data copy on CPU	`transpose([3,4], 0, 1)` → logical shape `[4,3]`, same memory	`Dense::forward` uses this to compute W^T without any allocation: `B().transpose(weights_.tensor, 0, 1)`
`cat(tensors, dim)`	Concatenate a list of tensors along one dimension — **does** allocate a new buffer	See below	MHA uses `cat` to merge attention heads after the per-head computation back into `[B, L, d_model]`

`cat` concrete example (two 3×2 matrices):

```
A:  |  1   2 |      B:  |  7   8 |
    |  3   4 |          |  9  10 |
    |  5   6 |          | 11  12 |
    shape [3, 2]        shape [3, 2]

cat([A, B], dim=0)  — append rows (more samples):       shape [6, 2]

    |  1   2 |
    |  3   4 |
    |  5   6 |
    |  7   8 |
    |  9  10 |
    | 11  12 |

cat([A, B], dim=1)  — append columns (more features):   shape [3, 4]

    |  1   2   7   8 |
    |  3   4   9  10 |
    |  5   6  11  12 |
```

`cat` along `dim=0` = "append more samples." `cat` along `dim=1` = "append more features." MHA uses `dim=1` on the head axis: after computing all h heads independently as `[B, L, d_k]` each, `cat(heads, dim=2)` along the feature axis reassembles them into `[B, L, h*d_k] = [B, L, d_model]`.

Transpose layout note. The CPU stride-swap trick means `B().transpose(W, 0, 1)` is O(1) — it creates a new `Tensor` header pointing to the same `data_` buffer but with `strides_` reversed. `CpuBackend::matmul` then handles the col/row-major identity described in Chapter 2. A naïve CUDA backend that assumes contiguous row-major layout before calling `cublasSgemm` would need to materialise a copy here — which is why the cuBLAS wrapper must accept leading-dimension arguments rather than transposing physically.

§D.7 — Synchronisation

Method	Behaviour
`sync()`	Block the calling thread until all pending operations on this backend's device are complete. **CpuBackend:** no-op — all CPU operations are synchronous by definition. **CudaBackend (planned):** `cudaDeviceSynchronize()`. **RemoteBackend (planned):** flush the gRPC stream and wait for the last server acknowledgement. Call `sync()` before reading a CUDA tensor on the CPU side, or before checkpointing mid-training.

§D.8 — Layer Backend-Readiness Map

Practical question: if `CudaBackend` were registered today via `setActive("cuda")`, which parts of the engine would actually accelerate — and which would silently stay on CPU?

Every layer's `forward()` and `backward()` path is analysed by whether it reaches the metal via `B()` or bypasses it with `flat(i)` loops.

Layer / component	Vtable coverage	Raw CPU bypasses	Net CUDA benefit if plugged in today
`Dense::forward`	`matmul`, `transpose`	Bias `flat(i)` loop (tiny)	~100% — GEMM paid for; bias loop negligible
`Dense::backward`	`matmul`, `transpose`	Same bias loop	~100%
`MultiHeadAttention::forward`	`matmul`, `transpose`, `divScalar`, `mulScalar`, `addScalar`	Softmax and causal-mask inner loops	Projections (dominant) fully accelerated; attention softmax stays CPU
`SigmoidFn::forward + backward`	`neg`, `exp`, `addScalar`, `div_`, `mul`, `subScalar`, `mulScalar`	None	Fully accelerated
`ReLUFn::forward`	`clamp`	None	Accelerated

Layer / component	Vtable coverage	Raw CPU bypasses	Net CUDA benefit if plugged in today
`ReLUFn::backward`	`mul`	Mask-build `flat(i)` loop	`mul` accelerated; mask build stays CPU
`TanhActFn::forward`	None	Full `std::tanh` loop	**Not accelerated**
`TanhActFn::backward`	`mul` , `mulScalar` , `subScalar`	None	Backward accelerated; forward still CPU
`SoftmaxFn::forward`	None	Full max/exp/sum loop	**Not accelerated**
`SoftmaxFn::backward`	None	Full dot-product loop	**Not accelerated**
`GELUFn::forward + backward`	None	Full tanh-polynomial loop	**Not accelerated**
`LeakyReLUFn::forward + backward`	None	Full `flat(i)` loops	**Not accelerated**
`MSE / BCE / CrossEntropy / Huber`	`sub` , `mul` , `mulScalar` , `mean` , `log` , `clamp` , `div_`	None	Fully accelerated
`BatchNorm1d`	`mul` , `mulScalar` , `addScalar`	Mean/variance `flat(i)` loops	Partially accelerated
`LayerNorm`	`mul` , `addScalar` , `mulScalar`	Per-row mean/rstd loops	Partially accelerated
`Dropout::forward`	None	Bernoulli + scale loop	**Not acceleratable** (CPU PRNG; GPU would need a cuRand kernel)
`Adam::step`	None	Per-parameter `flat(i)` loops	**Not accelerated** (could be rewritten as vtable calls)
`Embedding::forward`	None	Row-copy `flat(i)` loop	**Not accelerated** (needs `gatherRows` — not yet in vtable)
`Embedding::backward`	None	Scatter-accumulate `flat(i)` loop	**Not accelerated** (needs `scatterAddRows`)

Phase 4 vtable additions that would unlock the ❌/⚠️ rows:

Planned method	Unlocks
`gatherRows(table, indices)`	`Embedding::forward`
`scatterAddRows(dest, indices, src)`	`Embedding::backward`
`applyFunctor1D(tensor, fn)`	`TanhAct` , `GELU` , `LeakyReLU` , `Softmax` forwards (ADR-034 step 2)
`bernoulliMask(shape, p)`	`Dropout` GPU sampling

§D.9 — Future-Proofing: Adding Methods Without Breaking Existing Backends

The problem: Every `virtual ... = 0` in `IBackend` is a pure-virtual function. Adding a new pure-virtual after the interface is published breaks every backend that does not implement it — including third-party plugin backends that have no knowledge of the change.

Three patterns are available; NNStudio will use all three for different categories of new functionality.

Pattern 1 — Non-pure virtual with CPU fallback (recommended for new accelerations)

```
// IBackend.h — new method, not pure-virtual:
virtual Tensor gatherRows(const Tensor& table, const Tensor& indices) {
    // Default impl: CPU loop — old backends keep working
    Tensor out({indices.numel(), table.shape()[1]});
    for (int64_t i = 0; i < indices.numel(); ++i) {
        int64_t row = static_cast<int64_t>(indices.flat(i));
        for (int64_t c = 0; c < table.shape()[1]; ++c)
            out.flat(i * table.shape()[1] + c) =
                table.flat(row * table.shape()[1] + c);
    }
    return out;
}
```

Old backends keep working at CPU speed. A CUDA backend overrides with a `cub::DeviceGather` call. The caller does not need any conditional logic — it just calls `B().gatherRows(...)`. This is the pattern for the Phase 4 additions (`gatherRows`, `scatterAddRows`, `applyFunctor1D`).

Pattern 2 — Capability query flags (for hardware-specific or optional ops)

```
enum class BackendCap : uint32_t {
    GatherScatter = 1 << 0,   // gatherRows + scatterAddRows available as GPU kernels
    Functor1D     = 1 << 1,   // applyFunctor1D available (KAN/SIREN, ADR-034 step 2)
    HalfPrecision = 1 << 2,   // float16 arithmetic supported natively
    QuantumGEMM   = 1 << 3,   // quantum-accelerated matrix multiply (Phase 6+)
};

// In IBackend.h — default: no capabilities
virtual uint32_t capabilities() const noexcept { return 0; }
```

Callers check before using the fast path:

```
if (B().capabilities() & static_cast<uint32_t>(BackendCap::GatherScatter))
    out = B().gatherRows(table, indices);    // fast GPU kernel
else
    out = gatherRowsFallback(table, indices); // CPU reference path
```

This is how the CUDA driver itself works — `cudaDeviceGetAttribute` fills a capability struct before any kernel is launched.

Pattern 3 — API version gate on the backend descriptor (for mandatory new ops)

```
// In IBackend.h
virtual uint32_t apiVersion() const noexcept { return 1; }
```

`BackendRegistry::registerBackend()` checks `apiVersion()`. If a loaded plugin backend reports version 1 but the engine requires version 2 (because a new pure-virtual was added), the backend is rejected with a clear diagnostic:

```
[BackendRegistry] WARN: backend "mycuda" api_version=1, required=2 — reject
```

This mirrors `NNSTUDIO_PLUGIN_API_VERSION` in the plugin ABI (§10.7). Used for mandatory interface evolution only — prefer Pattern 1 for optional accelerations.

Adopted strategy for NNStudio (in line with ADR-034):

Class of change	Pattern	Target version
Current arithmetic primitives	mandatory pure-virtual	v1 (current)
`gatherRows` / `scatterAddRows`	non-pure + CPU fallback (Pattern 1)	v1.1 (Phase 4)
`applyFunctor1D` for KAN/SIREN	non-pure + CPU fallback (Pattern 1)	v2 (Phase 4, ADR-034 step 2)
`bernoulliMask` for GPU Dropout	non-pure + CPU fallback (Pattern 1)	v1.1 (Phase 4)
Any breaking mandatory change	version bump (Pattern 3)	v2+

§D.10 — Backend Compatibility Matrix

Legend: ✅ implemented / fully accelerated on that device — not a fallback
🟡 works but no hardware acceleration — equivalent to CPU
❌ not supported / not applicable
🔮 speculative / requires active hardware research
(planned) — method does not yet exist in `IBackend.h`

> `CudaBackend`, `RemoteBackend`, and `QuantumBackend` do not yet exist in the codebase. Entries for those columns document design intent and expected implementation complexity, not current reality.

`IBackend` method	`CpuBackend` *(Phase 1)*	`CudaBackend` *(Phase 4 plan)*	`RemoteBackend` *(Phase 5 plan)*	`QuantumBackend` *(Phase 6+)*
`name()` / `device()`	✅ `"cpu"` / `CPU`	✅ `"cuda"` / `CUDA`	✅ `"remote"` / `CPU`	✅ `"quantum"` / `QUANTUM`
`alloc(count)`	✅ `new float[]`	✅ `cudaMallocManaged`	🟡 remote buffer handle	🔮 quantum memory register
`to(tensor)`	🟡 ref-count no-op if same device	✅ `cudaMemcpy H→D / D→H`	✅ gRPC tensor serialisation	🟡 classical copy to/from device
`matmul(a, b)`	✅ Eigen row/col-major trick	✅ `cublasSgemm` / `cublasGemmEx`	✅ gRPC dispatch → remote CUDA	🔮 quantum GEMM (research 2024+)
`add(a, b)`	✅ Eigen element-wise	✅ element-wise CUDA kernel	✅ remote dispatch	🟡 classical fallback
`sub(a, b)`	✅ Eigen	✅ CUDA kernel	✅ remote	🟡
`mul(a, b)`	✅ Eigen (Hadamard)	✅ CUDA kernel	✅ remote	🟡
`div_(a, b)`	✅ loop	✅ CUDA kernel	✅ remote	🟡
`addScalar` / `subScalar` / `mulScalar` / `divScalar`	✅ loop	✅ `cublasSaxpy` / Thrust	✅ remote	🟡
`neg(a)`	✅ `mulScalar(-1)`	✅ fused into any element-wise kernel	✅ remote	🟡
`exp(a)`	✅ `std::exp` loop	✅ `cuDNN` / Thrust `exp`	✅ remote	🔮
`log(a)`	✅ `std::log` loop	✅ `cuDNN` / Thrust `log`	✅ remote	🔮
`sqrt(a)`	✅ `std::sqrt` loop	✅ Thrust `sqrt`	✅ remote	🔮
`abs(a)`	✅ loop	✅ Thrust `abs`	✅ remote	🟡
`clamp(a, lo, hi)`	✅ loop	✅ Thrust `clamp`	✅ remote	🟡
`sum(a, dim, keepdim)`	✅ Eigen reduce	✅ `cub::DeviceReduce::Sum`	✅ remote	🟡
`mean(a, dim, keepdim)`	✅ Eigen reduce	✅ `cub` + scale	✅ remote	🟡

`IBackend` method	`CpuBackend` *(Phase 1)*	`CudaBackend` *(Phase 4 plan)*	`RemoteBackend` *(Phase 5 plan)*	`QuantumBackend` *(Phase 6+)*
`max(a, dim, keepdim)`	✅ loop	✅ `cub::DeviceReduce::Max`	✅ remote	🟡
`reshape(a, shape)`	✅ zero-copy strides	✅ zero-copy if contiguous; copy if strided	✅ shape metadata only	🟡
`transpose(a, d0, d1)`	✅ strides swap, O(1)	🟡 may need contiguous copy for `cublasSgemm` leading-dim	✅ remote	🟡
`cat(tensors, dim)`	✅ allocate + copy	✅ `cudaMemcpy` segments	✅ remote	🟡
`sync()`	🟡 no-op (CPU is synchronous)	✅ `cudaDeviceSynchronize()`	✅ gRPC stream flush	🟡 no-op
`gatherRows` *(planned v1.1)*	🟡 Pattern 1 CPU fallback	✅ `cub::DeviceGather` / `cudaMemcpy` scatter	✅ remote	🟡
`scatterAddRows` *(planned v1.1)*	🟡 Pattern 1 CPU fallback	✅ atomic `scatter_add` kernel	✅ remote	🟡
`bernoulliMask` *(planned v1.1)*	🟡 Pattern 1 CPU fallback (std PRNG)	✅ `cuRAND` device kernel	✅ remote	❌ not applicable
`applyFunctor1D` *(planned v2, ADR-034 step 2)*	🟡 Pattern 1 CPU lambda call	🔮 JIT PTX / NVRTC kernel (research)	✅ remote	❌ not applicable

Reading the table:

- Every ✅ in `CudaBackend` is a place where `setActive("cuda")` gives a net speedup with **zero layer code change**.
- Every 🟡 in `CudaBackend` is a missed acceleration — layers using those methods would stay on CPU even with CUDA registered.
- The `transpose` 🟡 for CUDA is a known trade-off: CPU transpose is free (strides swap); CUDA cuBLAS expects contiguous row-major input, so a physical transpose copy may be needed unless the caller passes `CUBLAS_OP_T` directly — a `CudaBackend::matmul` implementation should absorb this internally via `ld` (leading dimension) arguments rather than materialising the copy.
- `applyFunctor1D` 🔮 in CUDA reflects that applying arbitrary C++ lambdas on the GPU requires JIT compilation (NVRTC / PTX). This is solvable but non-trivial, and is intentionally deferred to ADR-034 step 2.

§D.11 — Writing Plugin Activations That Use Vtable Primitives

This section is a practical guide for **plugin activation authors**: how to build an `IActivation` implementation that routes its compute through `IBackend` vtable calls so that a CUDA (or any future) backend accelerates it automatically, and how the framework surfaces acceleration warnings when the backend cannot honour those calls at hardware speed.

D.11.1 — The `B()` accessor

Every `IActivation` subclass has access to the active backend through `B()` (documented in §D.1). Calling any vtable method through `B()` routes computation to whatever backend is registered at runtime:

```cpp
// Plugin file: my_activation.cpp
#include <nnstudio/plugin-api/IActivation.h>

class SwishAct final : public nnstudio::IActivation {
public:
    // forward(x) = x * sigmoid(x) = x / (1 + exp(-x))
    Tensor forward(const Tensor& x) override {
        // All three calls dispatch through IBackend vtable:
        Tensor neg_x  = B().neg(x);              // §D.3 — arithmetic
        Tensor e      = B().exp(neg_x);           // §D.4 — math function
        Tensor denom  = B().addScalar(e, 1.0f);  // §D.3 — scalar arithmetic
        Tensor sig    = B().div_(x, denom);       // §D.3 — division (reuse x buffer)
        return B().mul(x, sig);                   // §D.3 — Hadamard product
    }

    Tensor backward(const Tensor& x, const Tensor& grad) override {
        // swish'(x) = swish(x) + sigmoid(x)(1 - swish(x))
        Tensor sw     = forward(x);
        Tensor neg_x  = B().neg(x);
        Tensor e      = B().exp(neg_x);
        Tensor denom  = B().addScalar(e, 1.0f);
        Tensor sig    = B().div_(Tensor::ones_like(x), denom);  // σ(x)
        Tensor one_sw = B().subScalar(B().mulScalar(sw, -1.0f), -1.0f); // 1 - sw(x)
        Tensor dswish = B().add(sw, B().mul(sig, one_sw));
        return B().mul(grad, dswish);
    }
};
```

Because every operation is a vtable call, this `SwishAct` is **fully backend-agnostic**. On `CpuBackend` it runs Eigen loops; on a future `CudaBackend` every call becomes a CUDA kernel with no code change in this file.

D.11.2 — Declaring the acceleration profile

`IActivation` exposes an optional virtual method that plugins can override to tell the framework exactly which backend ops they rely on:

```cpp
// IActivation.h (planned v1.1 addition — Pattern 1: non-pure with default)
struct BackendAccelerationProfile {
    bool fullyVtableDispatched = false;    // true → 0 raw loops, 100% vtable
    std::vector<std::string> vtableOps;    // list of IBackend methods called
    std::vector<std::string> rawLoopPaths; // names of any remaining raw loops
};

class IActivation {
public:
    virtual BackendAccelerationProfile backendAccelerationProfile() const {
        // Default: unknown — plugin did not declare
        return {}; // fullyVtableDispatched=false, empty lists
    }
};
```

A fully vtable-dispatched plugin overrides this:

```cpp
BackendAccelerationProfile SwishAct::backendAccelerationProfile() const override {
    return {
        .fullyVtableDispatched = true,
        .vtableOps = {"neg", "exp", "addScalar", "div_", "mul",
                      "subScalar", "mulScalar", "add"},
        .rawLoopPaths = {}
    };
}
```

The macro `NNSTUDIO_ACTIVATION_VTABLE_PROFILE(ops...)` is a convenience wrapper that generates this override automatically:

```
// Example macro use (defined in plugin-api/Macros.h):
NNSTUDIO_ACTIVATION_VTABLE_PROFILE(neg, exp, addScalar, div_, mul,
                                   subScalar, mulScalar, add)
```

D.11.3 — What the framework does with the profile

At registration time (`ActivationRegistry::registerActivation`), the engine calls `backendAccelerationProfile()` and stores the result alongside the factory. It uses the profile for three purposes:

1. Studio UI badge (see TODO.md → Properties panel → Backend acceleration status badge)

Profile result	Badge	Tooltip
`fullyVtableDispatched = true`	🟢 **Fully accelerated**	"All ops route through IBackend vtable; will benefit from CUDA / future backends."
`fullyVtableDispatched = false` , `rawLoopPaths` non-empty	🟡 **Partial — CPU loops**	"Raw loop paths: `{list}` . Performance is CPU-bound regardless of backend."
Default (no override)	🟣 **Unknown — plugin**	"Plugin did not declare its backend profile. Assume CPU-only performance."

2. Runtime log warning when backend mismatches profile

When the user switches to `CudaBackend` at training time, the dispatcher checks each activation in the `ComputeGraph` :

```
[BackendDispatch] WARN: activation "TanhAct" — raw loop paths present: ["forward flat(i) loop",
  "backward mask-build loop"]. These paths will NOT benefit from CUDA acceleration.
  Consider replacing with a vtable-only implementation. (See blueprints.md §D.11)
```

3. Export manifest annotation

When exporting a `.nnsx` bundle, each activation's acceleration profile is written into `manifest.json` :

```
"activations": [
  {
    "name": "SwishAct",
    "fullyVtableDispatched": true,
    "vtableOps": ["neg", "exp", "addScalar", "div_", "mul"]
  }
]
```

This lets a deployment tool (or CI check) refuse a build that depends on unaccelerated activations when targeting a high-throughput backend.

D.11.4 — CPU fallback: it is always there, no action needed

The question for plugin authors is not "how do I provide a CPU fallback?" — the `CpuBackend` is the fallback. Every vtable call transparently becomes a CPU Eigen operation on `CpuBackend` . The only question is **whether the active backend can accelerate those calls beyond CPU speed**.

The three-tier model:

Scenario	What happens
`CpuBackend` active, activation uses vtable only	✅ CPU Eigen path — correct and fast for CPU
`CudaBackend` active, activation uses vtable only	✅ CUDA kernel path — automatically accelerated

Scenario	What happens
`CudaBackend` active, activation has raw `flat(i)` loops	🟡 Loop runs on CPU-side data; effective backend speed = CPU; WARN logged
No backend registered (impossible — `CpuBackend` is always the fallback)	N/A

The framework guarantees `B()` is always non-null; plugin authors do not need guard clauses.

D.11.5 — Porting an existing raw-loop activation to vtable

The `/builtin/activations/` activations that currently have raw loops (§D.8) are the reference examples for this migration. The pattern:

```
BEFORE (raw loop — not accelerated):
    void forward(Tensor& out, const Tensor& in) {
        for (int i = 0; i < in.numel(); ++i)
            out.flat(i) = std::tanh(in.flat(i));   // CPU-only, no vtable
    }

AFTER (vtable — automatically accelerated):
    Tensor forward(const Tensor& x) override {
        // tanh(x) = (exp(2x) - 1) / (exp(2x) + 1)
        Tensor t2x = B().mulScalar(x, 2.0f);
        Tensor e   = B().exp(t2x);
        Tensor num = B().subScalar(e, 1.0f);
        Tensor den = B().addScalar(e, 1.0f);
        return B().div_(num, den);
    }
```

Note: `tanh` can also be expressed as `1 - 2 / (exp(2x) + 1)`, which uses one fewer allocation. Both are numerically correct; the second is slightly more allocation-efficient. In practice, once `IBackend` gains a native `tanh` vtable entry (Phase 4), the whole body becomes `return B().tanh(x);`.

Summary checklist for plugin activation authors:

- ☐ Implement `forward()` / `backward()` using only `B().someMethod(...)` calls (zero `flat(i)` loops).
- ☐ Override `backendAccelerationProfile()` (or use `NNSTUDIO_ACTIVATION_VTABLE_PROFILE(ops...)`).
- ☐ Do *not* add a CPU fallback — `CpuBackend` is the framework's built-in fallback.
- ☐ Test on `CpuBackend`; a future CI lane will test on `CudaBackend` (Phase 4).
- ☐ If a raw loop is unavoidable (e.g. custom indexing), declare it in `rawLoopPaths` so the UI and export manifest reflect reality.

Vocabulary

All short-hand identifiers used in this document, grouped by category.

A — C++ field names (engine source code)

Trailing `_` is the project convention for private member variables.

Symbol	Type	Lives in	Meaning
`data_`	`std::shared_ptr<float[]>`	`Tensor`	The flat heap array of floats that stores all values
`shape_`	`std::vector<int64_t>`	`Tensor`	Logical dimensions, e.g. `{3, 2}` = 3 rows × 2 columns
`strides_`	`std::vector<int64_t>`	`Tensor`	Navigation map: `strides_[k]` = how many floats to skip to move +1 in dimension k

Symbol	Type	Lives in	Meaning
`numel_`	`int64_t`	`Tensor`	Cached product of all `shape_` dimensions = total element count
`grad_`	`std::optional<Tensor>`	`Tensor`	Sibling tensor holding accumulated gradients; absent until first backward pass
`W_`	`Parameter`	`Dense`	Weight matrix parameter; `W_.tensor` has shape `{outF, inF}`
`b_`	`Parameter`	`Dense`	Bias parameter; `b_.tensor` has shape `{outF}`
`lastInput_`	`Tensor`	`Dense`, `ReLU`, `LeakyReLU`, `GELU`	Input saved during `forward()` so `backward()` can compute `dW = gradOut^T @ lastInput_` (Dense), or the sign-mask derivative (ReLU group)
`lastOutput_`	`Tensor`	`Sigmoid`, `TanhAct`, `Softmax`	Output saved during `forward()` because the derivative of these functions is expressible purely in terms of the output value — cheaper than recomputing from input
`layers_`	`std::vector<LayerPtr>`	`Sequential`	Ordered list of owned child layers
`training_`	`bool`	`ILayer`	True = train mode (Dropout active, BatchNorm uses batch stats); false = eval mode
`built_`	`bool`	`ILayer`	True after `build()` has allocated weight matrices; guards against double-allocation
`m_`	`std::unordered_map<…, Tensor>`	`Adam`	First-moment (mean gradient) running average per parameter
`v_`	`std::unordered_map<…, Tensor>`	`Adam`	Second-moment (mean squared gradient) running average per parameter
`t_`	`int64_t`	`Adam`	Step counter; used in bias-correction exponents

B — Math notation (standard ML convention)

Symbol	Reads as	Meaning
`W`	"weights"	The weight matrix of a Dense layer; shape `[outF, inF]`
`b`	"bias"	The bias vector of a Dense layer; shape `[outF]`
`x` / `X`	"input"	Input to a layer; uppercase = batch `[B, inF]`, lowercase = single sample `[inF]`
`y` / `Y`	"output"	Output of a layer; uppercase = batch `[B, outF]`, lowercase = single sample `[outF]`
`@`	"matmul"	Matrix multiplication operator (NumPy/Python notation used in this document)
`^T`	"transposed"	Matrix transpose; swaps rows and columns
`L`	"loss"	Scalar loss value — a single float measuring total prediction error for one step
`dW`	"delta W"	Shorthand for the gradient of W (dL/dW); same shape as W
`db`	"delta b"	Shorthand for the gradient of b (dL/db); same shape as b
`dX`	"delta X"	Gradient passed downward to the previous layer; same shape as X
`g_t`	"gradient at step t"	Raw gradient value at Adam step t; input to the moment update equations
`m_t`	"first moment"	Adam's running mean of gradients (momentum); smooths noisy gradient updates
`v_t`	"second moment"	Adam's running mean of squared gradients; enables per-weight adaptive learning rates
`beta_1`	"beta one"	Adam decay rate for first moment; typically 0.9
`beta_2`	"beta two"	Adam decay rate for second moment; typically 0.999
`alpha`	"alpha" / "learning rate"	Step size scalar; scales how far each update moves in parameter space

Symbol	Reads as	Meaning
`epsilon`	"epsilon"	Small constant (e.g. 1e-8) added to denominator to prevent division by zero
`theta`	"theta"	Generic parameter symbol; stands for any W or b in the optimizer update rule

C — Dimension shorthands (used in examples and diagrams)

Symbol	Expands to	Meaning
`B`	batch size	Number of samples processed simultaneously in one forward/backward call
`inF`	`inFeatures`	Number of input values arriving at a layer per sample
`outF`	`outFeatures`	Number of output values produced by a layer per sample
`M` , `N` , `K`	matrix dims	Generic matrix dimension names used in GEMM descriptions: `C[M,N] = A[M,K] @ B[K,N]`

D — Didactic shorthands (invented for this document)

These identifiers appear only in explanatory examples, not in production source code.

Symbol	Context	Meaning
`h1`	ONNX node name example	Hypothetical node `"h1"` in an ONNX graph — represents a hidden-layer output
`h1_biased`	ONNX node name example	Same node after the bias Add op; shows how `y = W*x + b` becomes two ONNX nodes
`x1` , `x2`	Dense 2->3 example	First and second features of a 2-feature input sample
`w00` , `w10` , …	Dense 2->3 example	Individual weight entries: w_{neuron, input}
`y0` , `y1` , `y2`	Dense 2->3 example	Individual neuron pre-activation outputs
`b0` , `b1` , `b2`	Dense 2->3 example	Bias values for each neuron
`x(0)` , `x(1)`	Batching example	Superscript = sample index within a batch

E — Standard library and industry terms

Term	Meaning
`GEMM`	**GEneral Matrix Multiply** — the Level 3 BLAS routine name. Full form: $C \leftarrow \alpha \cdot \mathrm{op}(A) \cdot \mathrm{op}(B) + \beta C$, where `op ∈ {N, T, C}` (no-op / transpose / conjugate-transpose) and α, β are scalar multipliers. Plain matrix product is GEMM with $\alpha = 1$, $\beta = 0$. The `S` in `cublasSgemm` = single-precision float; `D` = double; `H` = half. In NNStudio dispatched via `IBackend::matmul` ; underlies `Dense` , `Conv2D` (via im2col Phase 4), and every attention head in `MultiHeadAttention` . See §D.2 for the full spec.
`noalias()`	Eigen hint: output buffer does not overlap any input — skips defensive internal allocation
Row-major / C-order	Layout where the last index changes fastest; elements of one row are contiguous. Used by NNStudio `Tensor` .
Col-major / Fortran-order	Layout where the first index changes fastest. Used internally by Eigen.
`Result<T>`	Engine error-or-value return type. Holds either a `T` value or an error. Methods: `.value()` , `.error()` , `.hasValue()` .
`LayerPtr`	`std::shared_ptr<ILayer>` — ownership handle for a layer inside `Sequential::layers_`
`Shape`	`std::vector<int64_t>` — alias for tensor dimension lists throughout the engine

Term	Meaning
`Parameter`	`struct { std::string name; Tensor tensor; bool frozen; }` — the unit the Optimizer reads and updates
Bias-correction	Adam technique: dividing `m_t` / `v_t` by `(1 - beta^t)` counteracts near-zero initialisation at training start
Weight pruning	Post-training zeroing or removal of near-zero weights to reduce inference cost — not yet implemented
Transfer learning	Using a pre-trained model's weights as starting point, training only certain layers (the "head")
`frozen`	Flag on a `Parameter` : Optimizer skips its update; layer still computes and backprops its gradient
ONNX	Open Neural Network Exchange — standard format for exporting models as a flat graph of named op-nodes (MatMul, Add, Relu, ...)
DAG	Directed Acyclic Graph — general model topology enabling skip connections and multi-head architectures (cf. `Sequential` which is linear)
BCE	Binary Cross-Entropy loss. For one sample: `L = -[y*log(p) + (1-y)*log(1-p)]`
BPE	**Byte Pair Encoding** — subword tokenisation algorithm. Iteratively merges the most frequent adjacent byte/character pair in the training corpus into a new vocabulary token. Result: rare words split into known sub-units, common words stay whole. NNStudio ships a built-in BPE tokenizer plugin (319-token vocab).
im2col	"Image to column" — standard technique for expressing convolution as a single GEMM call: the input patches that each filter kernel would slide over are re-arranged into the columns of a matrix, so `output = filter_matrix × im2col_matrix` . Turns Conv2D into 100 % `IBackend::matmul` . Phase 4 item.
ABI	**Application Binary Interface** — the machine-level contract between compiled modules: calling convention, name mangling, struct layout, vtable slot order. NNStudio's plugin ABI is the C `nnstudio_plugin.h` interface; it is deliberately C (not C++) because the C ABI is stable across compilers.
PTX	**Parallel Thread Execution** — NVIDIA's virtual ISA / assembly language for CUDA kernels. `nvcc` compiles `.cu` → PTX → SASS (native GPU machine code). Relevant for `applyFunctor1D` JIT plan in ADR-034 step 2.

F — ML framework and hardware ecosystem

Term	What it is	Relation to NNStudio
CUDA	NVIDIA's parallel computing platform and programming model. Extends C/C++ with GPU kernel launch syntax (`<<<grid, block>>>`), unified memory, and a device driver. The *platform* that cuBLAS/cuDNN/Thrust all sit on top of.	`CudaBackend` (Phase 4) targets CUDA. The `Device::CUDA` enum tag and `§D.10` compatibility matrix document the planned integration.
cuBLAS	NVIDIA's CUDA implementation of the BLAS standard. `cublasSgemm` = single-precision GEMM on GPU — the key call inside `CudaBackend::matmul` . Prefix legend: `S` =float32, `D` =float64, `H` =float16, `C` =complex32.	Every `IBackend::matmul` call routes here on a CUDA backend. See `§D.2` .
cuDNN	NVIDIA's CUDA Deep Neural Network library. Provides GPU-accelerated implementations of conv, pooling, activation functions (tanh, sigmoid, relu, softmax), batch normalisation, and attention.	Phase 4: `CudaBackend::exp` , `CudaBackend::log` , etc. map to cuDNN element-wise ops. See `§D.4` CUDA column.
Thrust	NVIDIA's C++ parallel algorithms library (ships with CUDA). Provides `thrust::transform` , `thrust::reduce` , `thrust::sort` , etc. — GPU equivalents of `std::transform` / `std::reduce` .	Phase 4: `CudaBackend` element-wise and reduction methods (abs, clamp, sum, max) use Thrust.
Eigen	C++ header-only linear algebra template library. NNStudio's `CpuBackend` uses `Eigen::Map` to wrap raw `float*` buffers and call `Eigen::internal::gebp_kernel` (SGEMM). Eigen is col-major internally;	The only third-party math dependency in Phase 1.

Term	What it is	Relation to NNStudio
	`CpuBackend` uses the row/col transposition trick (Chapter 2) to stay row-major.	
BLAS	**Basic Linear Algebra Subprograms** — a 1979 Fortran API, now the universal standard for dense linear algebra. Three levels: L1 = vector-vector (`dot` , `axpy`), L2 = matrix-vector (`gemv`), L3 = matrix-matrix (`gemm`). Every ML framework's CPU backend implements BLAS, often via OpenBLAS or MKL.	Eigen's SGEMM is a BLAS Level 3 implementation. `IBackend::matmul` is conceptually BLAS L3 `dgemm` / `sgemm` .
PyTorch	Facebook/Meta's open-source ML framework. Dynamic computation graph ("define-by-run"), autograd, and `torch.nn` layer API. NNStudio's `torch_compat` layer is intentionally API-compatible with `torch.nn.Module` .	`torch_compat.h` maps `torch::nn::Linear` → `Dense` , `torch::nn::Sequential` → `Sequential` , etc. See §10.6.
LibTorch	The C++ distribution of PyTorch — the same headers and `.so` / `.dll` libraries used by the Python package, exposed as a standalone C++ SDK. Used when NNStudio is built with `-DENABLE_LIBTORCH=ON` .	Optional dependency; when present, `torch_compat` wraps LibTorch types directly instead of NNStudio's own.
Keras	Originally a standalone high-level DL API (François Chollet, 2015); now integrated into TensorFlow as `tf.keras` . Defines the `model.compile / .fit / .evaluate` API contract.	NNStudio's Python bridge exposes a `nnstudio.keras` façade: `keras.Sequential` , `.compile()` , `.fit()` , `.layers.*` . See §10.9.
TensorFlow	Google's open-source ML framework; computation graph is static ("define-then-run" in TF1, eager in TF2). `tf.keras` is the canonical high-level API.	Not a dependency; `nnstudio.runners.tf_serving` is an *inference client* that talks to a running TF Serving instance.
ONNX Runtime	Microsoft's cross-platform inference engine for `.onnx` models. Reads a model graph exported from PyTorch/TensorFlow/NNStudio and executes it — no training.	`nnstudio.runners.onnx_runtime` wraps the ONNX Runtime Python SDK as a `RunnerBase` implementation.
TF Serving	TensorFlow's production model server. Loads a `SavedModel` and exposes gRPC + REST inference endpoints.	`nnstudio.runners.tf_serving` sends inference requests to a running TF Serving endpoint.
Triton Inference Server	NVIDIA's high-performance multi-model inference server. Supports ONNX, TensorRT, PyTorch, TensorFlow, and custom backends; dynamic batching and concurrent model execution.	`nnstudio.runners.triton` is an inference client; Triton itself is not bundled.
KServe	Kubernetes-native model serving platform (CNCF project). Provides a standardised `InferenceService` CRD; backends can be Triton, TorchServe, TF Serving, etc.	`nnstudio.runners.kserve` targets the KServe v2 inference protocol.
gRPC	Google's open-source RPC framework. Uses HTTP/2 + Protocol Buffers. NNStudio uses gRPC for two purposes: `RemoteBackend` (dispatch tensor ops to a remote CUDA worker) and `nnstudio.runners.*` inference clients.	`RemoteBackend` (Phase 5) serialises `Tensor` to protobuf and sends via gRPC stream; `sync()` flushes the stream.
pybind11	Header-only C++11 library for creating Python extension modules. Exposes C++ classes/functions to Python with minimal boilerplate.	`python-bridge/bindings/module.cpp` uses pybind11 to expose `Tensor` , `Sequential` , `Dense` , `Trainer` , etc. to Python.
OpenAI API	REST API for hosted LLM inference (ChatGPT, GPT-4, etc.). Standard HTTP endpoint that accepts JSON prompt objects.	`nnstudio.runners.openai` wraps the OpenAI client as a `RunnerBase` — the model runs remotely, NNStudio handles the prompt/response contract.